

**ANALISIS PERFORMA *COMPLEMENT NAIVE BAYES* DAN *XGBOOST*
DALAM MENGATASI *CONCEPT DRIFT* PADA KLASIFIKASI *SPAM*
EMAIL MENGGUNAKAN PENDEKATAN *DOMAIN ADAPTATION***

LEMBAR PERSEMBAHAN

LEMBAR PERNYATAAN KEASLIAN SKRIPSI

**LEMBAR PERNYATAAN PERSETUJUAN PUBLIKASI KARYA
ILMIAH**

LEMBAR PERSETUJUAN DAN PENGESAHAN SKRIPSI

LEMBAR PEDOMAN PENGGUNAAN HAK CIPTA

KATA PENGANTAR

ABSTRAKSI

DAFTAR ISI

BAB I PENDAHULUAN.....	1
1.1. Latar Belakang Masalah.....	1
1.2. Rumusan Masalah.....	3
1.3. Tujuan dan Manfaat.....	4
1.3.1. Tujuan Penelitian	4
1.3.2. Manfaat Penelitian	4
1.4. Hipotesis	5
1.5. Batasan Masalah.....	6
BAB II LANDASAN TEORI	9
2.1. Tinjauan Pustaka.....	9
2.1.1. <i>Email Dan Spam</i>	9
2.1.2. <i>Machine Learning</i>	10
2.1.3. <i>Complement Naive Bayes (CNB)</i>	10
2.1.4. <i>Extreme Gradient Boosting (XGBoost)</i>	12
2.1.5. <i>Domain Adaptation dan Instance Weighting</i>	15
2.1.6. <i>TF-IDF (Term Frequency-Inverse Document Frequency)</i>	16
2.1.7. <i>Confusion matrix</i>	18
2.1.8. <i>Natural Language Processing (NLP) dan Text Preprocessing</i>	20
2.1.9. <i>Evolusi Spam dan Tantangan Concept Drift</i>	21
2.1.10. <i>Karakteristik Linguistik dan Anatomi Spam Modern</i>	21
2.1.11. <i>Rekayasa Fitur Heuristik dan Deteksi Platform Sah</i>	22
2.1.12. <i>Seleksi Fitur Uji Chi-Square (χ^2)</i>	24
2.1.13. <i>N-Grams dalam Representasi Sekuensial Teks</i>	24
2.1.14. <i>Bahasa Pemrograman Python</i>	25
2.1.15. <i>Lingkungan Pengembangan Visual Studio Code (VS Code)</i>	25
2.1.16. <i>Pustaka (Library) Pemrosesan Data dan Machine Learning</i>	26
2.2. Penelitian Terkait	27
BAB III METODOLOGI PENELITIAN	30
3.1. Proses dan Langkah Penelitian	30
3.1.1. <i>Kerangka Penelitian</i>	31
3.2. Metode Pengolahan dan Analisis Data	36

3.2.1. Metode Pengolahan Data	36
3.2.2. Analisis Data	37
BAB IV HASIL DAN PEMBAHASAN.....	39
4.1. Hasil Penelitian	39
4.1.1. Hasil Pengumpulan dan Distribusi Data Lintas Domain.....	39
4.1.2. Hasil Rekayasa Fitur dan Pembentukan Matriks <i>Hybrid</i>	40
4.1.3. Implementasi Seleksi Fitur dan <i>Instance Weighting</i>	41
4.1.4. Hasil Pelatihan Model dan Optimasi <i>Threshold</i>	43
4.2. Hasil Pengujian.....	44
4.2.1. Hasil Pengujian Metode 1 (Tanpa Adaptasi)	44
4.2.2. Hasil Pengujian Metode 2 (<i>Domain Adaptation</i>)	48
4.2.3. Perbandingan Performa Metode 1 dan Metode 2	51
4.2.4. Analisis Tambahan terhadap Rasio Data Adaptasi	53
4.2.5. Pembahasan Dampak <i>Domain Adaptation</i> terhadap <i>Concept Drift</i>	56
4.3. Implementasi Model dalam Bentuk Demo Aplikasi <i>Web</i> Lokal.....	58
4.3.1. Tujuan Pengembangan Demo Aplikasi.....	59
4.3.2. Lingkungan Implementasi Demo Aplikasi	59
4.3.3. Fitur dan Fungsionalitas Demo Aplikasi	60
4.3.4. Alur Kerja Demo Aplikasi	61
4.3.5. Tampilan Demo Aplikasi	62
4.3.6. Pembahasan Implementasi Demo Aplikasi	69
BAB V PENUTUP	72
5.1. Kesimpulan	72
5.2. Saran.....	73

DAFTAR GAMBAR

Gambar II. 1 Ilustrasi Additive training pada Arsitektur <i>XGBoost</i>	14
Gambar II. 2 Tahapan Text <i>Preprocessing</i> pada Data Email	20
Gambar III. 1 Kerangka Penelitian (Metode 1)	31
Gambar III.2 Kerangka Penelitian Domain Adaptation (Metode 2)	31
Gambar IV. 1 Grafik Top 20 Fitur <i>Chi-Square</i> (Metode 1).....	42
Gambar IV. 2 Grafik Top 20 Fitur <i>Chi-Square</i> (Metode 2).....	43
Gambar IV. 3 <i>Confusion matrix Complement Naive Bayes</i> Metode 1	46
Gambar IV. 4 <i>Confusion matrix XGBoost</i> Metode 1.....	47
Gambar IV. 5 Perbandingan Metrik Evaluasi Metode 1	48
Gambar IV. 6 <i>Confusion matrix Complement Naive Bayes</i> Metode 2	49
Gambar IV. 7 <i>Confusion matrix XGBoost</i> Metode 2.....	50
Gambar IV. 8 Perbandingan Metrik Evaluasi Metode 2	51
Gambar IV. 9 Perbandingan Akurasi Metode 1 dan Metode 2	52
Gambar IV. 10 Eksperimen Rasio Akurasi	53
Gambar IV. 11 Eksperimen Rasio <i>F1-score</i>	54
Gambar IV. 12 Eksperimen Rasio <i>FalsePositive</i>	55
Gambar IV. 13 Eksperimen Rasio <i>FalseNegative</i>	55
Gambar IV. 14 Halaman Utama Demo Aplikasi	63
Gambar IV. 15 Tampilan Prediksi Email Berbasis Teks	63
Gambar IV. 16 Tampilan Pengujian <i>Dataset CSV</i>	64
Gambar IV. 17 Tampilan Proses Pelatihan dan Evaluasi	65
Gambar IV. 18 Tampilan Hasil Evaluasi Metode 1	66
Gambar IV. 19 Tampilan Hasil Evaluasi Metode 2.....	66
Gambar IV. 20 Tampilan Perbandingan Metode 1 dan Metode 2	67

Gambar IV. 21 Tampilan Pengujian Model Hasil Pelatihan	68
Gambar IV. 22 Tampilan Riwayat Eksperimen.....	69

DAFTAR TABEL

Tabel II.1 Perbandingan Penelitian Terkait	27
Tabel IV. 1 Skenario Metode 1	39
Tabel IV. 2 Skenario Metode 2	40
Tabel IV. 3 Hasil Pengujian Metode 1 (Tanpa Adaptasi)	45
Tabel IV. 4 Hasil Pengujian Metode 2 (Domain Adaptation)	48
Tabel IV. 5 Perbandingan Akurasi Metode 1 dan Metode 2	52
Tabel IV. 6 Hasil Eksperimen Variasi Rasio Data Adaptasi	53

DAFTAR LAMPIRAN

BAB I

PENDAHULUAN

1.1. Latar Belakang Masalah

Email tetap menjadi pilar utama komunikasi digital dalam ekosistem profesional maupun personal di seluruh dunia. Seiring dengan peningkatan volume lalu lintas *email*, ancaman *spam* juga mengalami eskalasi yang signifikan, di mana pesan-pesan tersebut kini bertransformasi menjadi instrumen serangan siber yang canggih. Serangan modern sering kali memanfaatkan teknik rekayasa sosial dan penyebaran *malware* melalui tautan *phishing* yang dirancang agar terlihat sah di mata sistem keamanan tradisional. Kompleksitas ancaman ini menuntut adanya sistem klasifikasi otomatis yang tidak hanya statis, tetapi juga adaptif dalam membedakan antara *Email* sah dan *Email spam* secara akurat (Tian et al., 2025). Pada awalnya, *spam* umumnya hanya berupa pesan promosi yang mengganggu, namun saat ini pesan *spam* sering kali mengandung tautan *phishing* maupun *malware* yang dirancang untuk menipu pengguna dan bahkan mampu mengelabui sistem keamanan. Kondisi tersebut menuntut adanya sistem klasifikasi yang mampu membedakan secara otomatis antara *Email* yang sah dan *Email spam* (Tusher et al., 2024).

Dalam ranah komputasi, pendekatan *Machine Learning* telah menjadi standar emas untuk menangani klasifikasi teks karena kemampuannya dalam mengekstraksi pola dari data dalam jumlah besar. Berbagai algoritma telah diuji, mulai dari model probabilistik seperti *Complement Naive Bayes* yang efisien pada data teks, hingga algoritma *ensemble* generasi modern seperti *Extreme Gradient Boosting* (*XGBoost*) yang mampu menangani interaksi fitur *non-linear* secara optimal. Banyak

penelitian melaporkan tingkat akurasi yang sangat tinggi, bahkan melampaui 94%, saat model dilatih menggunakan *Dataset* publik historis seperti Enron-Spam atau korpus Kaggle lainnya. Namun, performa yang terlihat impresif pada data historis tersebut sering kali mengalami penurunan yang drastis ketika dihadapkan pada data *email* pribadi di lingkungan nyata saat ini (Manguma & Fatra, 2024).

Penurunan performa yang signifikan ini disebabkan oleh fenomena yang dikenal sebagai *Concept Drift*. *Concept Drift* merepresentasikan pergeseran distribusi statistik data di mana karakteristik kosakata *spam* masa lalu berbeda total dengan taktik *spam* masa kini. *Dataset* historis dari awal tahun 2000-an tidak merekam munculnya tipe *email* sah modern seperti notifikasi *One-Time Password* (OTP), resi belanja daring, atau peringatan keamanan akun yang kini mendominasi kotak masuk pengguna. Akibatnya, terjadi kesenjangan domain (*domain gap*) yang lebar antara data pelatihan (*source domain*) dan data pengujian aktual (*target domain*). Ketidaksinkronan ini menyebabkan model yang dilatih murni pada data lama menjadi kurang mampu melakukan generalisasi, bahkan sering kali menghasilkan akurasi yang hanya berada di kisaran 50% hingga 60% saat diuji pada *email* personal masa kini (Huang et al., 2023).

Untuk mengatasi kegagalan model statis tersebut tanpa harus melakukan pelabelan data baru secara lebih besar yang memakan waktu dan biaya, metode *Domain Adaptation* menawarkan pendekatan yang relevan secara metodologis. Teknik ini memungkinkan model untuk melakukan kalibrasi ulang batas keputusan klasifikasinya dengan cara menyuntikkan sebagian kecil sampel dari domain target ke dalam proses pelatihan. Melalui implementasi teknik *Instance Weighting*, sampel data modern diberikan bobot matematis yang lebih tinggi dalam perhitungan fungsi kerugian (*loss function*) algoritma. Secara teknis, intervensi ini mengarahkan

algoritma untuk memprioritaskan pola dari domain baru sambil tetap mempertahankan pengetahuan dasar dari domain lama, sehingga performa model berpotensi meningkat setelah sebagian data dari domain target dimasukkan ke dalam proses pelatihan. (Rafiee Sevyeri, 2022).

Berdasarkan kesenjangan performa yang nyata antara model tradisional dan tuntutan data modern, penelitian ini dirancang untuk membedah masalah tersebut melalui analisis komparatif yang ketat. Fokus utama penelitian ini adalah membandingkan dua skenario eksperimen: Metode 1 (Model Murni) yang melatih model murni menggunakan data Kaggle, dan Metode 2 (*Domain Adaptation*) yang menggunakan teknik pembobotan instans untuk menjembatani pergeseran data. Dengan menguji algoritma *Complement Naive Bayes* dan *XGBoost* pada kedua skenario ini, penelitian ini bertujuan menganalisis secara empiris peran adaptasi domain dalam meningkatkan performa model klasifikasi *spam* pada *Email* modern. (Akpan et al., 2026).

1.2. Rumusan Masalah

1. Bagaimana performa algoritma *Complement Naive Bayes* dan *Extreme Gradient Boosting (XGBoost)* yang dilatih menggunakan *Dataset* historis dalam menghadapi fenomena *Concept Drift* pada klasifikasi *spam email*
2. Bagaimana pengaruh penerapan *Domain Adaptation* melalui skema injeksi data primer dan teknik *Instance Weighting* terhadap performa algoritma *Complement Naive Bayes* dan *Extreme Gradient Boosting (XGBoost)* dalam mengatasi pergeseran karakteristik kosakata *spam*, serta konfigurasi adaptasi yang paling optimal?

3. Bagaimana perbandingan performa klasifikasi antara model tanpa adaptasi dan model dengan adaptasi domain berdasarkan metrik akurasi, presisi, *recall*, dan *F1-score*?

1.3. Tujuan dan Manfaat

1.3.1. Tujuan Penelitian

1. Menganalisis performa algoritma *Complement Naive Bayes* dan *Extreme Gradient Boosting (XGBoost)* yang dilatih menggunakan *Dataset* historis dalam menghadapi fenomena *Concept Drift* pada klasifikasi *spam email*.
2. Menguji pengaruh penerapan *Domain Adaptation* melalui skema injeksi data primer dan teknik *Instance Weighting* terhadap performa algoritma *Complement Naive Bayes* dan *Extreme Gradient Boosting (XGBoost)* dalam mengatasi pergeseran karakteristik kosakata *spam*, serta menentukan konfigurasi adaptasi yang paling optimal.
3. Menganalisis perbandingan hasil evaluasi antara model tanpa adaptasi dan model dengan adaptasi domain untuk menentukan pendekatan dengan performa terbaik berdasarkan metrik akurasi, presisi, *recall*, dan *F1-score*.

1.3.2. Manfaat Penelitian

1. Bagi Penulis
 - a. Sebagai salah satu syarat kelulusan bagi penulis untuk memperoleh gelar sarjana (S1) pada program studi Informatika di Universitas Bina Sarana Informatika.
 - b. Mengimplementasikan teori *Machine Learning* yang telah dipelajari ke dalam praktik nyata, serta memahami mekanisme penanganan *Concept Drift* melalui penggunaan *Dataset* modern.

- c. Memperdalam pemahaman teknis mengenai perbedaan karakteristik dan performa antara algoritma berbasis probabilistik dan algoritma berbasis *ensemble*.

2. Bagi Objek Penelitian

- a. Menyediakan referensi model klasifikasi yang optimal dan relevan dengan evolusi ancaman siber terkini untuk membedakan pesan *spam* dan *non-spam*.
- b. Membantu meningkatkan efisiensi dan keamanan penggunaan *Email* dengan meminimalisir gangguan dari pesan-pesan sampah atau komunikasi digital yang berbahaya.

3. Bagi Pembaca

- a. Menjadi rujukan ilmiah mengenai perbandingan efektivitas *Complement Naive Bayes* dan *XGBoost*, serta pentingnya pembaruan *Dataset* dalam menjaga relevansi sistem deteksi *spam*.
- b. Memberikan dasar informasi dan inspirasi bagi peneliti selanjutnya untuk mengembangkan sistem filter *spam* yang lebih canggih menggunakan metode ekstraksi fitur yang berbeda.

1.4. Hipotesis

Hipotesis penelitian merupakan dugaan sementara yang diajukan sebagai jawaban atas rumusan masalah, di mana kebenarannya akan dibuktikan secara empiris melalui serangkaian proses pengujian data. Dalam penelitian ini, hipotesis disusun untuk memprediksi hasil perbandingan kinerja antara algoritma *Complement Naive Bayes* dan *XGBoost*, serta efektivitas penerapan intervensi adaptasi data. Berikut adalah rincian hipotesis yang diajukan:

H_0 : Tidak terdapat perbedaan performa yang signifikan antara algoritma *Complement Naive Bayes* dan *XGBoost*, serta tidak ada peningkatan akurasi dari penerapan metode *Domain Adaptation* dalam mengatasi fenomena *Concept Drift* pada klasifikasi *email* modern.

H_1 : Terdapat perbedaan performa yang signifikan antara algoritma *Complement Naive Bayes* dan *XGBoost*, di mana penerapan metode *Domain Adaptation* menunjukkan peningkatan performa algoritma dari dampak *Concept Drift* dan meningkatkan akurasi klasifikasi secara optimal.

1.5. Batasan Masalah

Untuk menjaga agar penelitian tetap terfokus pada tujuan yang telah ditetapkan serta memastikan kedalaman analisis terhadap objek yang diteliti, maka ditetapkan beberapa batasan masalah dalam penelitian ini sebagai berikut (Akpan et al., 2026):

1. Ruang Lingkup Data Latih (*Source Domain*): Menggunakan *Dataset spam* publik historis yang diekstraksi dari korpus Kaggle sebanyak 5.728 pesan *Email* berbahasa Inggris (terdiri dari 4.360 *non-spam* dan 1.368 *spam*).
2. Ruang Lingkup Data Uji (*Target Domain*): Menggunakan data primer berjumlah 1.000 pesan *Email* aktual yang diseimbangkan (*balanced*) menjadi 500 *Email spam* dan 500 *Email non-spam*, yang diekstraksi secara langsung dari kotak masuk (*inbox*) *Email* pribadi peneliti.
3. Skenario Eksperimen: Evaluasi komparatif dibatasi pada dua skenario utama, yaitu Metode 1 (Model Murni tanpa adaptasi) dan Metode 2 (*Domain Adaptation*) dengan skema injeksi 30% sampel data primer ke dalam korpus pelatihan.

4. Parameter Adaptasi Domain: Implementasi teknik *Domain Adaptation* menggunakan metode *Instance Weighting* dengan memberikan amplifikasi bobot matematis (*sample weight*) sebesar 8,0 pada instans data primer yang disuntikkan ke dalam set pelatihan.
5. Metode Ekstraksi Fitur: Fitur diekstraksi menggunakan kombinasi *Word TF-IDF* (*N-Gram* 1,2), *Character TF-IDF* (*N-Gram* 3,5), serta penambahan 57 fitur heuristik kustom untuk metode 1, dan 51 fitur heuristik kustom untuk metode 2 yang mencakup rasio kapitalisasi, tanda baca, fitur struktural teks, dan deteksi kata kunci (*keywords*) spesifik.
6. Algoritma Klasifikasi: Algoritma yang dibandingkan terbatas pada *Complement Naive Bayes* (dengan seleksi fitur *Chi-Square*) dan *Extreme Gradient Boosting* (*XGBoost*) yang didukung oleh akselerasi perangkat keras GPU (*Compute Unified Device Architecture*).
7. Optimasi Keputusan: Penentuan hasil akhir klasifikasi menggunakan teknik *Dynamic Threshold Tuning* pada rentang 0,10 - 0,95 untuk Metode 1 dan 0,15 - 0,85 untuk Metode 2 untuk mengoptimalkan perolehan metrik *F1-score*.
8. Metrik Evaluasi: Pengukuran performa model dilakukan berdasarkan matriks kebingungan (*Confusion matrix*) yang mencakup nilai akurasi, presisi, *recall*, dan *F1-score*.
9. Lingkungan Pengembangan (*Tools*): Proses eksperimen, pengolahan data, hingga pelatihan model dilakukan menggunakan bahasa pemrograman Python (versi 3.x) dengan Visual Studio Code (VS Code) sebagai Lingkungan Pengembangan Terintegrasi (*Integrated Development Environment*).
10. Batasan Luaran: Penelitian ini berfokus pada analisis performa model secara eksperimental. Demo aplikasi web lokal (*localhost*) yang ditampilkan pada BAB

IV hanya digunakan sebagai media demonstrasi tambahan untuk memperlihatkan penerapan model, bukan sebagai objek utama evaluasi maupun sistem produksi.

BAB II

LANDASAN TEORI

2.1. Tinjauan Pustaka

Tinjauan pustaka ini menguraikan secara mendalam berbagai teori, konsep, dan algoritma yang menjadi landasan ilmiah dalam proses klasifikasi *spam Email* pada penelitian ini. Pembahasan mencakup prinsip dasar pengolahan data teks, mekanisme kerja algoritma *Machine Learning* yang dipilih, serta metrik evaluasi yang digunakan sebagai parameter objektif untuk mengukur keberhasilan pengujian model.

2.1.1. *Email Dan Spam*

Email atau surat elektronik adalah protokol pertukaran pesan digital lintas jaringan yang menjadi sarana komunikasi absolut di era modern. Kendati efisien, *Email* menjadi target utama penyebaran *spam*, yaitu pesan massal yang tidak diinginkan (*unsolicited bulk Email*). Secara historis, *spam* didominasi oleh iklan komersial. Namun, dalam perkembangannya, *spam* bertransformasi menjadi vektor serangan siber (*cyberattack*) yang mendistribusikan *malware*, *ransomware*, dan tautan *phishing*. Serangan ini mengeksploitasi kerentanan psikologis pengguna melalui teknik rekayasa sosial (*social engineering*) untuk mencuri data kredensial atau informasi finansial (Tian et al., 2025).

Seiring berjalannya waktu, taktik rekayasa sosial ini mengalami evolusi kosakata yang sangat cepat. Penipu secara terus-menerus memodifikasi struktur kalimat, menyisipkan karakter tersembunyi, dan merekayasa ulang narasi untuk menghindari deteksi *filter* keamanan. Perubahan distribusi statistik dari karakteristik *spam* ini dikenal dalam ilmu *Machine Learning* sebagai fenomena *Concept Drift*. *Concept Drift* menyebabkan algoritma yang dilatih menggunakan *Dataset* historis

menjadi usang, karena fitur tekstual yang sebelumnya valid untuk mendeteksi *spam* tidak lagi relevan dengan pola serangan modern. Hal ini memicu kemerosotan akurasi yang drastis saat algoritma dihadapkan pada lalu lintas jaringan yang aktual (Huang et al., 2023).

2.1.2. *Machine Learning*

Machine Learning adalah pergeseran paradigma dari pemrograman eksplisit (berbasis logika *If-Then*) menjadi pemrograman berbasis pengenalan pola. Dalam ranah deteksi *spam*, pendekatan yang digunakan adalah *Supervised Learning* (pembelajaran terawasi), di mana algoritma diberikan kumpulan data historis yang telah dilabeli secara manual (kategori 1 untuk *spam*, 0 untuk *non-spam*) sebagai landasan kebenaran.

Komputer tidak memiliki kemampuan kognitif untuk membaca teks. Oleh karena itu, *Machine Learning* dalam klasifikasi teks bekerja dengan mengubah struktur bahasa manusia menjadi ruang vektor matematis. Tujuan utama dari algoritma klasifikasi seperti *Complement Naive Bayes* maupun *XGBoost* adalah menemukan fungsi matematis atau batas keputusan (*decision boundary*) yang paling optimal untuk memisahkan vektor data teks *spam* dari vektor data teks *non-spam* dengan tingkat kesalahan prediksi (*error rate*) seminimal mungkin. Algoritma tidak menganalisis makna kalimat, melainkan mengkalkulasi probabilitas, jarak, atau bobot fitur numerik di dalam ruang dimensi (Almarri, 2025).

2.1.3. *Complement Naive Bayes (CNB)*

Sebelum menganalisis *Complement Naive Bayes* (CNB), perlu dipahami kelemahan struktural dari algoritma standarnya, yakni *Multinomial Complement Naive Bayes* (MNB). Algoritma *Complement Naive Bayes* tradisional beroperasi berdasarkan Teorema Bayes dengan asumsi independensi yang kuat antar fitur (*naivety*). Pada

klasifikasi teks konvensional, MNB menghitung probabilitas bahwa sebuah dokumen milik suatu kelas berdasarkan frekuensi kemunculan kata. Namun, MNB memiliki kelemahan fatal ketika dihadapkan pada kumpulan data yang distribusinya tidak seimbang (*imbalanced data*). Dalam kondisi tersebut, MNB cenderung membiaskan prediksinya secara berlebihan ke arah kelas mayoritas, karena kelas yang memiliki lebih banyak dokumen secara otomatis memiliki probabilitas *prior* yang lebih besar (Manguma & Fatra, 2024).

Untuk mengatasi bias statistik tersebut, pendekatan *Complement Naive Bayes* (CNB) diimplementasikan. CNB adalah modifikasi langsung dari MNB yang dirancang secara khusus untuk klasifikasi teks yang mengalami ketidakseimbangan kelas atau frekuensi fitur yang ekstrem (seperti pada teks *spam* yang dipenuhi kata kunci repetitif).

Asumsi utama yang mendasari algoritma ini disebut sebagai *Conditional Independence*, di mana setiap kata dalam sebuah dokumen dianggap berdiri sendiri dan tidak memiliki kaitan dengan kata lainnya. Meskipun asumsi ini sering kali dianggap terlalu menyederhanakan realitas bahasa (*naive*), namun algoritma ini sangat tangguh dalam klasifikasi teks karena alasan berikut:

1. Efisiensi pada Data Berdimensi Tinggi: Data teks biasanya memiliki ribuan fitur (kosakata unik). *Complement Naive Bayes* mampu mengolah data berskala besar ini dengan beban komputasi yang sangat ringan dibandingkan algoritma berbasis pohon atau saraf.
2. Kesesuaian dengan Distribusi Frekuensi: Algoritma ini bekerja sangat baik dengan model *Bag-of-Words*, di mana kehadiran atau frekuensi kata tertentu (seperti "hadiah" atau "gratis") menjadi indikator probabilistik yang kuat untuk menentukan kelas *spam*.

3. Kebutuhan Data Latih yang Sedikit: *Complement Naive Bayes* dapat mencapai performa yang stabil meskipun hanya dilatih dengan jumlah data yang relatif terbatas dibandingkan model kompleks lainnya (Firmansyah et al., 2025).

Alih-alih menghitung probabilitas bahwa sebuah dokumen milik suatu kelas, CNB menghitung probabilitas bahwa dokumen tersebut bukan milik kelas tertentu. Bobot untuk sebuah kata (t_i) pada kelas (c) dihitung berdasarkan frekuensinya di luar kelas (c):

$$w_{ci} = \log \left(\frac{N_{ti} + \alpha}{N_t + \alpha \cdot d} \right)$$

Di mana N_{ti} adalah jumlah kemunculan kata pada dokumen yang tidak berada di kelas c , dan α merupakan parameter pemulusan (*smoothing*). Normalisasi bobot ini membuat CNB secara empiris jauh lebih stabil dan akurat dalam menangani korpus teks yang mengalami ketidakseimbangan kelas akibat *Concept Drift* (Manguma & Fatra, 2024).

2.1.4. *Extreme Gradient Boosting (XGBoost)*

Extreme Gradient Boosting (XGBoost) adalah algoritma *ensemble learning* berbasis pohon keputusan (*decision tree*) yang beroperasi di bawah kerangka kerja *gradient boosting*. Secara fundamental, *XGBoost* tidak membangun model secara independen, melainkan menggunakan prinsip *Additive training*. Dalam proses ini, model-model baru ditambahkan secara iteratif untuk memprediksi residu atau kesalahan (*error*) yang dihasilkan oleh model-model sebelumnya, yang kemudian digabungkan untuk menghasilkan prediksi final yang lebih akurat (Gladyza et al., 2025).

Keunggulan ilmiah utama yang membedakan *XGBoost* dari algoritma *boosting* konvensional terletak pada optimasi fungsi objektifnya. Fungsi ini secara

matematis menggabungkan fungsi kerugian (*loss function*) untuk mengukur kecocokan data dengan suku regulasi (*regularization term*) untuk mengendalikan kompleksitas model. Hal ini memungkinkan *XGBoost* secara otomatis menangani masalah *overfitting*, yang sering terjadi pada klasifikasi teks dengan dimensi fitur yang sangat tinggi (Lukito & Wahyuningrum, 2025).

Pemilihan *XGBoost* dalam penelitian ini didasarkan pada beberapa justifikasi teknis:

1. Kemampuan Menangkap Pola *Non-Linear*: Berbeda dengan *Complement Naive Bayes* yang mengasumsikan independensi antar fitur, *XGBoost* mampu mengidentifikasi hubungan *non-linear* dan interaksi kompleks antar kata yang sering digunakan oleh *spammer* untuk mengelabui filter pesan.
2. Optimasi Sistem dan Kecepatan: "*Extreme*" pada algoritma ini merujuk pada optimasi perangkat keras, termasuk pemrosesan paralel dan *cache-aware access*, yang memungkinkannya mengolah *Dataset Enron* berskala besar dengan efisiensi komputasi yang jauh melampaui algoritma *ensemble* tradisional (Gladyza et al., 2025).
3. Regulasi Terintegrasi: Dengan adanya dukungan regulasi L1 (*Lasso*) dan L2 (*Ridge*), algoritma ini secara implisit melakukan seleksi fitur selama proses pembangunan pohon. Hal ini sangat krusial saat bekerja dengan bobot *TF-IDF* untuk memastikan model hanya fokus pada fitur kata yang memiliki nilai diskriminatif paling tinggi (Dwipayoga & Raharja, 2025).
4. Fleksibilitas Metrik: *XGBoost* memungkinkan penyesuaian fungsi tujuan yang dapat dioptimalkan secara spesifik untuk meminimalisir *False Positive*, yang merupakan metrik paling kritis dalam skenario deteksi *Email spam*.

Keistimewaan matematis *XGBoost* terletak pada fungsi objektifnya yang secara eksplisit memuat parameter penalti (regulasi) untuk mencegah *overfitting*, yang dirumuskan sebagai berikut:

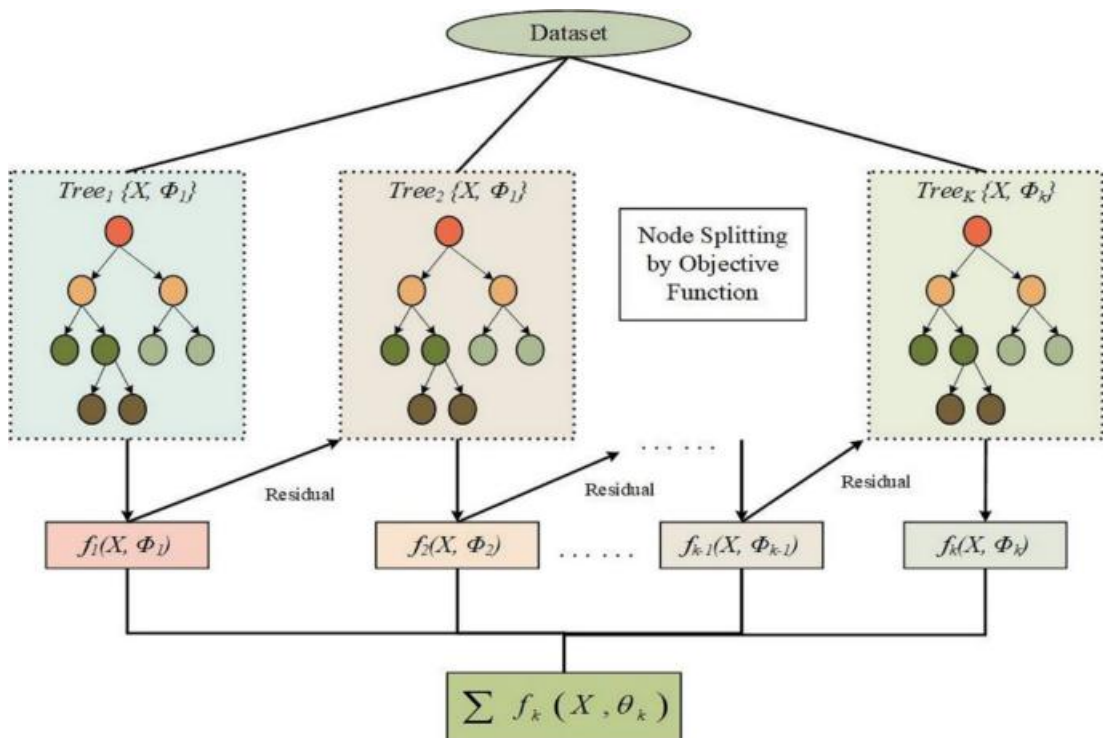
$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

Keterangan

$\mathcal{L}(\phi)$: Fungsi objektif yang dievaluasi.

$\sum_i l(\hat{y}_i, y_i)$: Fungsi kerugian (*loss function*) yang mengukur selisih antara nilai prediksi dengan nilai aktual dari *Email*.

$\sum_k \Omega(f_k)$: Suku regulasi (*L1/Lasso* atau *L2/Ridge*) yang mengontrol kompleksitas pohon keputusan untuk menghindari model yang terlalu rumit dan menghafal *noise*.



Gambar II. 1

Ilustrasi Additive training pada Arsitektur XGBoost

(Sumber : Khan et al., 2025)

2.1.5. Domain Adaptation dan Instance Weighting

Untuk mengurangi dampak dari *Concept Drift* tanpa harus melakukan pelabelan ulang pada puluhan ribu data baru, pendekatan *Domain Adaptation* diimplementasikan. *Domain Adaptation* adalah cabang dari *Transfer Learning* yang berfungsi menyelaraskan distribusi fitur antara domain sumber (*source domain* yang memuat data historis) dan domain target (*target domain* yang memuat data aktual). Dengan menyuntikkan sebagian kecil sampel dari domain target ke dalam korpus pelatihan, algoritma dipaksa untuk mengenali karakteristik linguistik dari lingkungan yang baru (Mir Dastagir Ali et al., 2026).

Penyuntikan sampel minoritas ke dalam korpus mayoritas sering kali diabaikan oleh algoritma pengklasifikasi. Oleh karena itu, pendekatan ini diintegrasikan dengan teknik *Instance Weighting* (Pembobotan Instans). Teknik ini memanipulasi fungsi kerugian (*loss function*) algoritma secara matematis dengan memberikan nilai bobot (w) yang lebih tinggi pada sampel domain target. Fungsi kerugian terbobot dirumuskan sebagai:

$$L_{\text{weighted}} = \sum [w_i \times l(y_i, f(x_i))]$$

Dalam formulasi tersebut, $l(y_i, f(x_i))$ adalah kerugian prediksi aktual, dan w_i adalah amplifikasi bobot. Dengan memberikan penalti asimetris (misalnya $w_i = 8.0$) pada sampel adaptasi, algoritma akan melakukan pembaruan batas keputusan (*decision boundary*) secara masif jika terjadi kesalahan klasifikasi pada sampel modern tersebut (Rafiee Sevyeri, 2022).

Dalam implementasinya, pendekatan *Supervised Domain Adaptation* memungkinkan pemanfaatan sebagian kecil sampel berlabel dari domain target selama proses pemodelan bersama dengan data dari domain sumber. Penggunaan proporsi data target dalam jumlah yang terbatas ini bertujuan untuk mengkalibrasi ulang batas

keputusan (*decision boundary*) model tanpa memicu terjadinya *overfitting* atau *catastrophic forgetting* terhadap karakteristik dasar dari domain asal. Pembatasan porsi data target pada proses pelatihan ini juga krusial untuk mempertahankan volume data uji yang lebih besar, sehingga metrik evaluasi akhir memiliki signifikansi statistik yang valid dalam merepresentasikan performa model di lingkungan operasional yang nyata (Zhuang et al., 2021)

2.1.6. *TF-IDF (Term Frequency-Inverse Document Frequency)*

TF-IDF bukan sekadar penghitungan kata biasa, melainkan metode pembobotan statistik yang dirancang untuk mengevaluasi signifikansi sebuah kata dalam sebuah dokumen terhadap seluruh kumpulan data (*corpus*). Fungsi utama *TF-IDF* dalam klasifikasi teks adalah sebagai filter informasi untuk memisahkan kata-kata bermakna dari kata-kata umum yang tidak memiliki nilai diskriminatif. Mekanisme ini terdiri dari dua komponen utama:

1. *Term Frequency* (TF): Mengukur intensitas sebuah kata dalam satu dokumen. Namun, frekuensi tinggi saja tidak menjamin kata tersebut penting (misalnya kata hubung "dan" atau "yang").
2. *Inverse Document Frequency* (IDF): Bertugas menekan bobot kata-kata yang muncul terlalu sering di seluruh dokumen dan menaikkan bobot kata-kata yang unik.

Alasan pemilihan *TF-IDF* dalam penelitian ini adalah:

- a. *Reduksi Noise*: Dalam *Dataset Enron*, banyak terdapat istilah bisnis umum yang muncul di hampir semua *Email*. *TF-IDF* secara otomatis akan memberikan bobot rendah pada kata-kata tersebut, sehingga algoritma *Complement Naive Bayes* dan *XGBoost* dapat fokus pada kata-kata yang benar-benar menjadi ciri khas *spam*.

- b. Representasi Numerik yang Presisi: Dibandingkan dengan *Count Vectorizer* yang hanya menghitung frekuensi, *TF-IDF* memberikan nilai desimal yang lebih halus yang mencerminkan kepentingan relatif suatu kata, sehingga membantu *XGBoost* dalam menemukan titik pemisah (*splitting point*) yang lebih akurat pada pohon Keputusan (Dwipayoga & Raharja, 2025).

Secara matematis, bobot akhir dari suatu kata dalam dokumen dihitung dengan mengalikan nilai *Term Frequency* (TF) dan *Inverse Document Frequency* (IDF) melalui persamaan berikut:

$$W_{t,d} = TF_{t,d} \times IDF_t$$

Keterangan:

$W_{t,d}$: Bobot akhir dari term t dalam dokumen d .

$TF_{t,d}$: Frekuensi kemunculan term t di dalam dokumen d .

Untuk mencegah bias terhadap kata yang sangat sering muncul di seluruh korpus *Email* (seperti kata hubung), nilai IDF dihitung menggunakan logaritma natural dengan persamaan:

$$IDF_t = \log \left(\frac{1 + N}{1 + df_{(t)}} \right) + 1$$

Keterangan :

N : Total keseluruhan dokumen (*Email*) dalam *Dataset*.

df_t : Jumlah dokumen yang mengandung term t di dalamnya.

Persamaan logaritmik ini memastikan bahwa kata yang muncul di hampir semua *Email* akan memiliki nilai IDF mendekati 0, sehingga bobot akhir akan menjadi sangat kecil dan algoritma dapat mengabaikan noise tersebut.

2.1.7. Confusion matrix

Dalam *Machine Learning*, performa model klasifikasi tidak bisa hanya dievaluasi berdasarkan jumlah tebakan yang benar secara global (*Akurasi*). *Confusion matrix* memberikan evaluasi komprehensif berupa tabel kontingensi yang memetakan hasil prediksi model berhadapan dengan label aktual data ke dalam empat kuadran: *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN).

Berdasarkan nilai dari keempat kuadran tersebut, performa algoritma *Complement Naive Bayes* dan *XGBoost* dalam penelitian ini dievaluasi menggunakan empat metrik matematis utama:

1. *Akurasi (Accuracy)* Akurasi mengukur persentase total prediksi yang benar (baik *spam* maupun *non-spam*) terhadap keseluruhan data. Meskipun umum digunakan, akurasi dapat memberikan kesimpulan yang menyesatkan jika distribusi *Dataset* tidak seimbang.

$$Akurasi = \frac{TP + TN}{TP + TN + FP + FN}$$

2. *Presisi (Precision)* Presisi mengukur proporsi prediksi *spam* yang benar-benar akurat dibandingkan dengan total tebakan *spam* oleh model. Dalam domain klasifikasi *Email*, metrik ini paling krusial. Nilai *False Positive* (FP) yakni *Email* pekerjaan atau dokumen penting yang salah diklasifikasikan sebagai *spam* memiliki risiko operasional yang sangat fatal. Oleh karena itu, sistem harus mengoptimalkan presisi setinggi mungkin.

$$Presisi = \frac{TP}{TP + FP}$$

3. *Recall (Sensitivity)* *Recall* mengukur proporsi *Email spam* aktual yang berhasil dideteksi dan dicegat oleh model. Nilai *False Negative* (FN) yakni pesan *spam*

yang bocor dan masuk ke kotak masuk utama memang mengganggu pengguna, *False Negative* tetap berisiko karena spam dapat masuk ke kotak masuk utama, namun dalam konteks kehilangan email penting, *False Positive* menjadi perhatian utama karena email sah dapat tertahan sebagai spam.

$$Recall = \frac{TP}{TP + FN}$$

4. *F1-score* adalah rata-rata harmonis antara Presisi dan *Recall*. Metrik ini digunakan untuk mengevaluasi model secara objektif ketika terdapat *trade-off* (tarik-ulur) antara menjaga agar pesan sah tidak masuk ke folder *spam* (Presisi tinggi) sekaligus memaksimalkan pencegahan pesan sampah (*Recall* tinggi).

$$F1 - Score = 2 \times \frac{Presisi \times Recall}{Presisi + Recall}$$

Pada penelitian ini, nilai presisi, *recall*, dan *F1-score* yang digunakan dalam evaluasi akhir dihitung menggunakan pendekatan *weighted average*. Pendekatan ini menghitung metrik pada setiap kelas, yaitu *spam* dan *non-spam*, kemudian memberikan bobot berdasarkan jumlah data pada masing-masing kelas. Dengan demikian, nilai evaluasi yang dihasilkan tidak hanya merepresentasikan performa model pada satu kelas positif saja, tetapi juga mencerminkan kemampuan model secara keseluruhan dalam mengklasifikasikan kedua kelas *email*.

Dalam pengklasifikasian lintas domain, mengandalkan ambang batas probabilitas statis (0.5) sering kali menghasilkan kelas prediksi yang bias. Oleh karena itu, pendekatan *Dynamic thresholding* diterapkan, di mana sistem mengevaluasi *F1-score* secara iteratif pada berbagai titik potong probabilitas (misalnya 0.10 hingga 0.95) untuk menemukan batas matematis yang paling optimal sebelum model melakukan pengambilan keputusan mutlak (Lukito & Wahyuningrum, 2025).

2.1.8. *Natural Language Processing (NLP)* dan *Text Preprocessing*

Dalam klasifikasi teks, algoritma *Machine Learning* tidak dapat secara langsung memproses teks mentah. Oleh karena itu, diperlukan *Natural Language Processing* (NLP) untuk membersihkan dan menormalkan data linguistik melalui tahap *Text Preprocessing* (Mantha, 2025).

Tahapan standar yang diimplementasikan dalam pemrosesan *email spam* meliputi:

1. *Text Normalization (Case Folding)*: Mengonversi seluruh karakter huruf menjadi huruf kecil (*lowercase*), serta menghapus tanda baca, angka, karakter khusus, dan *tag* HTML untuk memastikan keseragaman format data.
2. *Tokenization*: Memecah teks utuh dari badan *email* menjadi unit-unit kata individu atau *tokens*, sehingga fitur teks dapat diekstraksi dan dihitung frekuensinya secara terstruktur.
3. *Stopword Removal*: Menghapus kata-kata umum yang memiliki frekuensi kemunculan sangat tinggi namun tidak memberikan kontribusi signifikan secara semantik (seperti kata hubung atau preposisi), guna mengurangi dimensi *noise*.
4. *Stemming / Lemmatization*: Mengubah kata-kata yang memiliki imbuhan kembali ke bentuk dasar atau akar katanya. Proses ini krusial untuk mencegah duplikasi fitur dari kata yang bermakna sama namun memiliki struktur tata bahasa yang berbeda (Durga Prasad et al., 2026).



Gambar II. 2
Tahapan Text Preprocessing pada Data Email

2.1.9. Evolusi *Spam* dan Tantangan *Concept Drift*

Sistem deteksi *spam* modern memerlukan pemahaman konseptual tentang bagaimana anomali data berevolusi. *Email spam* saat ini tidak lagi sebatas iklan massal yang tidak diinginkan, melainkan telah menjadi vektor utama untuk serangan rekayasa sosial (*social engineering*) (Tusher et al., 2024). Secara linguistik, pesan *spam* sering kali menggunakan pola psikologis untuk memanipulasi emosi pengguna, seperti penggunaan frasa yang menjanjikan hadiah atau menciptakan urgensi palsu. Lebih lanjut, anatomi *spam* tingkat lanjut sering disisipi dengan tautan *phishing* atau *malware* tersembunyi yang bertujuan mengelabui korban agar menyerahkan kredensial (K. Meghna et al., 2025).

Perubahan taktik linguistik dan strategis dari para *spammer* dari waktu ke waktu memunculkan tantangan klasifikasi yang dikenal sebagai *Concept Drift* (Pergeseran Konsep) (Tusher et al., 2024). *Concept Drift* terjadi ketika probabilitas kondisional dari kelas (*spam/non-spam*) terhadap fitur masukan (teks) berubah seiring berjalannya waktu. Model klasifikasi yang dilatih menggunakan *Dataset* usang (misalnya dari awal tahun 2000-an) akan mengalami degradasi akurasi yang parah jika diterapkan pada *email* masa kini, karena kosakata (*vocabulary gap*) dan pola ancaman yang digunakan tidak lagi relevan. Oleh karena itu, pembaruan data latih menggunakan korpus kontemporer merupakan mitigasi paling efektif untuk mempertahankan ketangguhan model klasifikasi teks. (Akpan et al., 2026).

2.1.10. Karakteristik Linguistik dan Anatomi *Spam* Modern

Sistem deteksi *spam* modern memerlukan pemahaman konseptual tentang bagaimana anomali data berevolusi. *Email spam* saat ini tidak lagi sebatas iklan massal yang tidak diinginkan, melainkan telah menjadi vektor utama untuk serangan rekayasa sosial (*social engineering*). Secara linguistik, pesan *spam* sering kali menggunakan

pola psikologis untuk memanipulasi emosi pengguna, seperti penggunaan frasa yang menjanjikan hadiah ("gratis") atau menciptakan urgensi palsu ("penawaran terbatas", "mendesak") (Tusher et al., 2024).

Lebih lanjut, anatomi *spam* tingkat lanjut sering disisipi dengan tautan *phishing* atau *malware* tersembunyi yang bertujuan mengelabui korban agar menyerahkan kredensial (Reddy et al., 2026). Oleh karena itu, pemodelan klasifikasi menggunakan *Machine Learning* tidak hanya berpatokan pada frekuensi satu kata secara independen, melainkan mengevaluasi indikator kredibilitas domain dan representasi teks secara keseluruhan (Sahil Gedam & Prof. Tara Shende, 2026).

2.1.11. Rekayasa Fitur Heuristik dan Deteksi Platform Sah

Selain mengekstraksi bobot kata menggunakan TF-IDF, sistem deteksi *spam* modern memerlukan rekayasa fitur heuristik (*hand-crafted features*) untuk menangkap pola perilaku siber yang tidak berwujud kata. Rekayasa fitur ini mendelegasikan pengetahuan pakar (*domain knowledge*) ke dalam bentuk kalkulasi matematis yang dapat diproses oleh algoritma klasifikasi (Firmansyah et al., 2025).

Fitur heuristik tersebut dibagi ke dalam tiga kategori utama:

1. **Anomali Struktural dan Tanda Baca:** *Spammer* secara psikologis berusaha menarik perhatian korban atau menembus filter *regex* dengan memanipulasi struktur teks. Hal ini dikuantifikasi melalui perhitungan rasio huruf kapital (*ALL-CAPS ratio*), kepadatan tanda seru (!), tanda tanya (?), dan tanda dolar (\$). Penggunaan huruf kapital secara masif dan simbol mata uang yang padat memiliki korelasi statistik yang sangat tinggi terhadap serangan rekayasa sosial dan penipuan finansial. Selain itu, panjang *Email* yang dinormalisasi berdasarkan jumlah kata juga dihitung sebagai indikator tambahan, mengingat *Email spam* promosi cenderung memiliki panjang yang lebih besar dibandingkan *Email* notifikasi singkat.

2. Kepadatan Entitas Digital (URL, *Email*, dan HTML): *Email* sampah tingkat lanjut tidak lagi menggunakan teks polos (*plain text*), melainkan menyematkan banyak tautan (*link*), alamat kontak palsu, dan elemen HTML yang disembunyikan untuk mengarahkan pengguna ke situs *phishing*. Menghitung rasio kepadatan token URL dan token *Email* memberikan indikator yang kuat terhadap tingkat bahaya suatu *email*.
3. Deteksi Platform Sah (*Legitimate Platform Detection*): Fenomena *Concept Drift* menyebabkan *email* sah (*non-spam*) masa kini didominasi oleh notifikasi sistem otomatis, seperti peringatan keamanan atau pembaruan layanan. Untuk mencegah kesalahan klasifikasi (*False Positive*) pada *email* jenis ini, pendekatan *whitelist* heuristik diimplementasikan dengan mendeteksi penyebutan platform terpercaya (seperti Google, Facebook, Amazon, Github, dan Microsoft). *Email* yang mengandung entitas platform sah ini diberikan bobot probabilistik tambahan sebagai *email non-spam* (Tian et al., 2025).

Melalui penggabungan antara fitur TF-IDF (berbasis kosakata) dan matriks fitur heuristik (berbasis struktur dan perilaku), algoritma *ensemble* seperti *XGBoost* dapat menyusun batas keputusan (*decision boundary*) multidimensional yang jauh lebih tangguh terhadap pergeseran taktik *spam* masa kini.

Selain fitur struktural, sistem juga mengimplementasikan deteksi kata kunci *spam* (*spam keyword detection*) secara biner. 44 kata kunci (termasuk frasa multi-kata seperti "*buy now*", "*act now*", dll) untuk metode 1 dan 38 kata kunci untuk metode 2 yang secara universal diasosiasikan dengan *Email spam* seperti '*free*', '*urgent*', '*winner*', '*verify*', dan '*unsubscribe*' diperiksa keberadaannya dalam setiap *Email*. Setiap kata kunci menghasilkan nilai 1 (ada) atau 0 (tidak ada), membentuk vektor biner yang merepresentasikan intensitas penggunaan kosakata *spam*.

2.1.12. Seleksi Fitur Uji *Chi-Square* (χ^2)

Penggabungan ekstraksi vektor pada tingkat kata dan tingkat karakter secara otomatis akan memproduksi matriks berdimensi sangat tinggi (high-dimensional *Sparse matrix*). Ledakan dimensi ini dapat memicu fenomena curse of dimensionality yang merusak akurasi algoritma probabilistik. Untuk mereduksi noise, metode SelectKBest berbasis uji statistik *Chi-Square* (χ^2) diimplementasikan. Uji ini mengevaluasi tingkat independensi antara suatu fitur teks dengan label kelas (*spam* atau *non-spam*). Kalkulasi skor χ^2 dirumuskan sebagai:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$$

Di mana O_i adalah frekuensi kemunculan observasi aktual dari suatu token, dan E_i adalah frekuensi kemunculan yang diharapkan jika token tersebut tidak memiliki korelasi dengan label kelas. Fitur dengan nilai χ^2 tertinggi menunjukkan dependensi klasifikasi yang kuat, sehingga disaring dan dipertahankan dalam model (K. Meghna et al., 2025).

2.1.13. *N-Grams* dalam Representasi Sekuensial Teks

Meskipun ekstraksi fitur menggunakan pendekatan statistik tradisional sangat efisien, model sering kali kehilangan konteks mengenai urutan struktural kalimat. Untuk mengatasi kelemahan tersebut, tahap vektorisasi teks dapat dikombinasikan dengan teknik *N-Grams* (seperti *unigrams* atau *bigrams*) untuk menangkap informasi sekuensial dan makna frasa majemuk secara lebih utuh. Menggabungkan representasi fitur berbasis *TF-IDF* dengan analisis *N-Grams* terbukti dapat memperkaya ruang dimensi model, sehingga meningkatkan ketahanan (*robustness*) algoritma *ensemble* dalam memisahkan batas keputusan antara *email spam* dan *non-spam* (Adebanjo et al., 2025).

2.1.14. Bahasa Pemrograman Python

Python adalah bahasa pemrograman tingkat tinggi berorientasi objek yang ditafsirkan secara dinamis. Dalam domain kecerdasan buatan dan pemrosesan bahasa alami (NLP), Python telah menjadi standar industri karena sintaksisnya yang intuitif dan dukungan ekosistem sumber terbuka (*open-source*) yang masif. Pemilihan Python dalam penelitian ini didasarkan pada kemampuannya untuk memfasilitasi integrasi yang mulus antara tahapan prapemrosesan teks *email* yang kotor dengan eksekusi algoritma klasifikasi *Machine Learning* tingkat lanjut tanpa memerlukan komputasi tingkat rendah yang berbelit-belit (Harnadh, 2025).

2.1.15. Lingkungan Pengembangan *Visual Studio Code (VS Code)*

Visual Studio Code (VS Code) adalah *Integrated Development Environment (IDE)* berbasis teks dan editor kode sumber ringan yang dikembangkan oleh Microsoft. VS Code secara ekstensif digunakan dalam proyek sains data karena arsitekturnya yang modular dan dapat diperluas (*extensible*).

Dalam proses pengolahan data *email spam* pada penelitian ini, VS Code tidak hanya digunakan sebagai penyunting teks, melainkan sebagai pusat kendali komputasi. Kemampuan VS Code dalam mendukung integrasi *Jupyter Notebook* memungkinkan eksekusi skrip Python secara interaktif (*cell-by-cell*), sehingga peneliti dapat memvisualisasikan data, melakukan *debugging* pada tahap *Text Preprocessing*, dan memantau evaluasi model secara *real-time* dalam satu antarmuka yang kohesif (Sharma et al., 2025). Selain itu, fitur isolasi lingkungan virtual (*virtual environment*) pada VS Code memastikan bahwa seluruh dependensi modul eksternal yang digunakan berjalan stabil tanpa konflik sistem (Dushyant Kaushik, 2025).

2.1.16. Pustaka (*Library*) Pemrosesan Data dan *Machine Learning*

Arsitektur klasifikasi *spam* di dalam Python sepenuhnya digerakkan oleh sekumpulan pustaka (*library*) khusus pemrosesan data. Pustaka-pustaka esensial yang beroperasi di belakang layar selama proses rekayasa fitur dan pemodelan meliputi:

1. **Pandas:** Digunakan sebagai struktur data utama untuk membaca korpus mentah (seperti berkas CSV publik dan kotak masuk pribadi peneliti) dan mengonversinya menjadi *DataFrame tabular*. Pandas memfasilitasi manipulasi data berskala besar, pembersihan baris kosong (*handling missing values*), dan penataan label data secara efisien sebelum masuk ke tahap NLP (K. Meghna et al., 2025).
2. **NumPy (*Numerical Python*):** Merupakan fondasi komputasi ilmiah di Python yang memproses manipulasi matriks dan *array* multidimensi tingkat tinggi. Matriks bobot kata yang dihasilkan dari proses *TF-IDF* sangat bergantung pada kecepatan komputasi aljabar linear berbasis NumPy (K. Meghna et al., 2025).
3. **Scikit-Learn:** Pustaka *Machine Learning* komprehensif yang menyediakan modul arsitektural untuk seluruh pipa (*pipeline*) pemrosesan. Scikit-Learn mengeksekusi pembagian porsi data latih dan uji (*train_test_split*), ekstraksi vektor kata matematis (*TfidfVectorizer*), pemodelan klasifikasi *Complement Naive Bayes*, hingga kalkulasi metrik *Confusion matrix* (B.Sruthi et al., 2026).
4. **XGBoost Library:** Modul eksternal independen yang mengimplementasikan kerangka *Extreme Gradient Boosting*. Pustaka ini secara spesifik dioptimalkan untuk menangani vektor matriks teks jarang (*Sparse matrix*) berdimensi sangat tinggi, dengan pemanfaatan alokasi memori dinamis yang membuat kecepatan eksekusinya melampaui algoritma *ensemble* tradisional (Mir Dastagir Ali et al., 2026).

2.2. Penelitian Terkait

Penelitian terkait berisi rangkuman dari penelitian-penelitian sebelumnya yang relevan dengan topik klasifikasi *spam Email*. Hal ini bertujuan untuk memetakan posisi penelitian saat ini di antara penelitian yang sudah ada, serta mengidentifikasi celah penelitian (*research gap*) yang dapat dikembangkan. Ringkasan perbandingan antara penelitian terdahulu dengan penelitian yang sedang dilakukan dapat dilihat pada Tabel II.1 berikut:

*Tabel II.1
Perbandingan Penelitian Terkait*

No	Peneliti & Tahun	Judul	Metode	Hasil
1	(Hasan et al., 2021)	<i>An Analysis of Machine Learning Algorithms and Deep Neural Networks for Email Spam Classification using Natural Language Processing</i>	<i>Machine Learning Konvensional, XGBoost, Deep Neural Networks</i>	<i>XGBoost mencapai skor evaluasi tertinggi dibandingkan algoritma dasar lainnya pada Dataset statis..</i>
2	(Gladyza et al., 2025)	<i>Analisis Klasifikasi Spam Email Menggunakan Metode Extreme Gradient Boosting (XGBoost)</i>	<i>XGBoost, Stratified K-Fold Cross Validation, Confusion matrix</i>	<i>XGBoost mampu mencapai presisi yang sangat stabil dalam mencegah Email sah salah diklasifikasikan sebagai pesan sampah (False Positive).</i>
3	(Manguma & Fatra, 2024)	<i>Analisis Performa Algoritma Klasifikasi untuk Deteksi Spam pada Email</i>	<i>Complement Naive Bayes, K-Nearest Neighbor, Random Forest, SVM</i>	<i>Complement Naive Bayes memiliki tingkat presisi yang kompetitif dan efisien, menjadikannya standar dasar (baseline) yang kuat untuk deteksi spam.</i>
4	(Tian et al., 2025)	<i>Improving the accuracy of cybersecurity spam Email detection using ensemble techniques: A stacking approach Machine Learning</i>	<i>Stacking Ensemble, Machine Learning</i>	<i>Menunjukkan bahwa model tunggal sering gagal pada data siber yang kompleks tanpa optimasi fitur.</i>

No	Peneliti & Tahun	Judul	Metode	Hasil
		<i>for spam Email detection</i>		
5	(Akpan et al., 2026)	<i>Email Phishing Detection Using Machine Learning Approaches</i>	Klasifikasi Berbasis <i>Machine Learning</i>	Membuktikan perlunya analisis semantik dan ekstraksi fitur yang kuat karena serangan <i>phishing/spam</i> modern dirancang untuk mengelabui filter statis.
6	(Zhang et al., 2022)	<i>A Structure-Aware Method for Cross-domain Text Classification</i>	<i>Graph Attention Network (GAT), Correlation Alignment, Unsupervised Domain Adaptation</i>	Mampu menangkap hubungan korelasi struktural antar fitur kata menggunakan sub-graph dan menyelaraskan distribusi teks lintas domain guna meningkatkan akurasi klasifikasi secara signifikan.
7	(Li et al., 2026)	<i>Your Next State-of-the-Art Could Come from Another Domain: A Cross-Domain Analysis of Hierarchical Text Classification</i>	<i>Domain Adaptation, Pre-trained Language Models, Domain-Specific Fine-tuning</i>	Membuktikan bahwa melakukan adaptasi domain (<i>Domain Adaptation</i>) pada model klasifikasi teks spesifik secara signifikan meningkatkan kemampuan generalisasi model dibanding pendekatan statis.

Berdasarkan kelima penelitian terdahulu pada Tabel II.1, terlihat bahwa algoritma *Machine Learning* telah diuji secara ekstensif untuk menangani *spam Email*. Penelitian oleh (Manguma & Fatra, 2024) membuktikan bahwa algoritma *Complement Naive Bayes* memiliki ketangguhan pada tingkat presisi probabilistik. Di sisi lain, riset dari (Hasan et al., 2021) dan (Gladyza et al., 2025) mengonfirmasi superioritas komputasi arsitektur *XGBoost* dalam memproses ruang dimensi fitur teks yang besar. Namun, mayoritas penelitian tersebut masih berfokus pada pengujian dalam satu domain data yang sama, sehingga belum memetakan bagaimana performa model saat

menghadapi pergeseran karakteristik data di dunia nyata (Tian et al., 2025). Selanjutnya, efektivitas metode *Domain Adaptation* dalam mengatasi pergeseran distribusi teks lintas domain telah dibuktikan secara empiris oleh (Zhang et al., 2022) melalui penyelarasan fitur struktural, serta didukung oleh (Li et al., 2026) model dihadapkan pada lalu lintas *Email* personal modern yang memiliki distribusi kosakata berbeda. Tantangan arsitektural ini menyebabkan sistem deteksi yang ada saat ini cenderung tidak fleksibel terhadap evolusi ancaman siber (Jáñez-Martino et al., 2025).

Pembaharuan (*novelty*) utama dari penelitian ini adalah implementasi strategi mitigasi *Concept Drift* melalui eksperimen dua metode paralel. Berbeda dengan penelitian sebelumnya, riset ini tidak hanya mengukur performa algoritma *Complement Naive Bayes* dan *XGBoost* secara standar, tetapi menguji efektivitas intervensi *Domain Adaptation* menggunakan teknik *Instance Weighting*. Dengan menyuntikkan 30% sampel data primer dan memberikan amplifikasi bobot matematis sebesar 8,0, penelitian ini memberikan kontribusi baru berupa pembuktian empiris mengenai pemulihan performa model klasifikasi pada pengujian lintas domain (*cross-domain*) terhadap *Email* personal aktual (Akpan et al., 2026).

BAB III

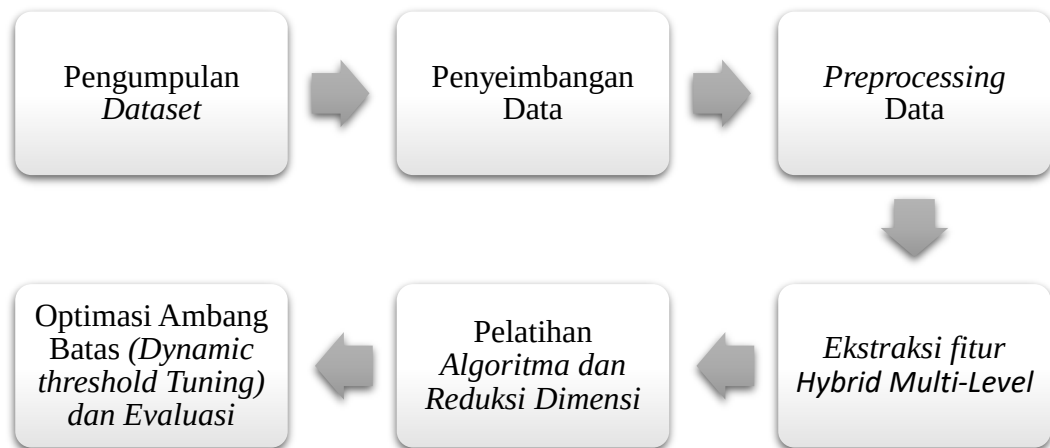
METODOLOGI PENELITIAN

3.1. Proses dan Langkah Penelitian

Penelitian ini dirancang menggunakan arsitektur eksperimental komparatif untuk mendemonstrasikan secara empiris dampak pergeseran konsep (*Concept Drift*). Sesuai dengan fokus penelitian, perbandingan utama difokuskan pada skenario pengolahan *Dataset* (pendekatan data), bukan sekadar perbandingan algoritma. Eksperimen komputasional dibagi ke dalam dua alur independen:

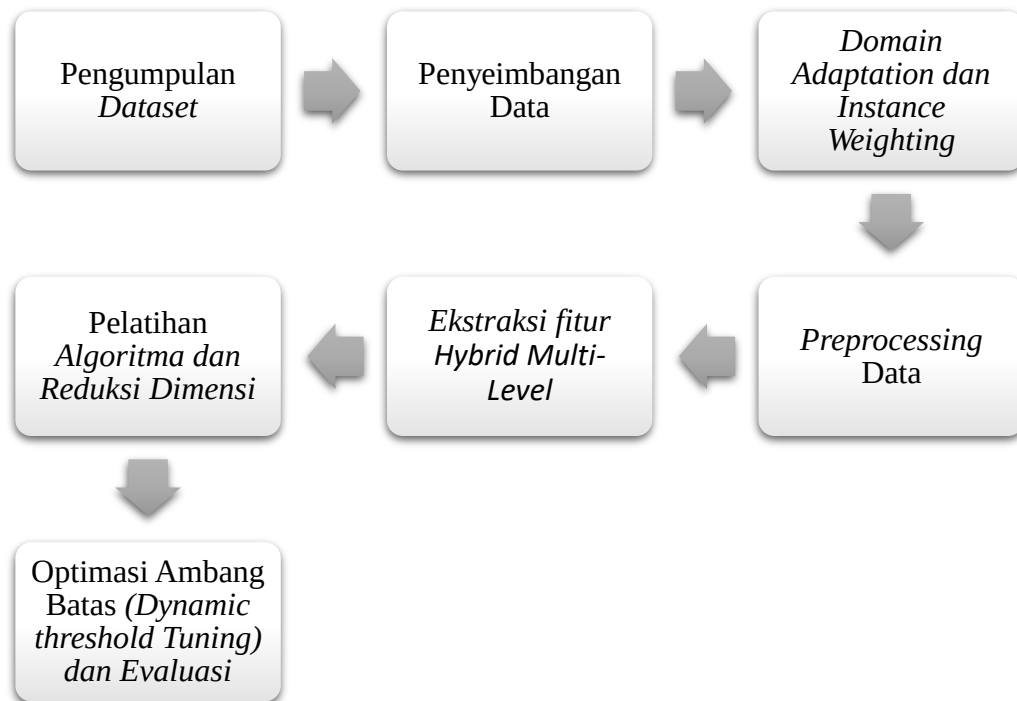
1. Metode 1 (Skenario Data Murni / Tanpa Adaptasi): Model dilatih secara eksklusif menggunakan *Dataset* historis (100% data sekunder Kaggle), kemudian dievaluasi kualitas generalisasinya secara langsung terhadap 1.000 data uji personal (data primer murni). Skenario ini merepresentasikan kondisi rentan di mana sistem hanya mengandalkan data usang.
2. Metode 2 (Skenario *Domain Adaptation*): Model dilatih menggunakan *Dataset* historis Kaggle yang dikombinasikan dengan 30% data primer sebagai data adaptasi, kemudian dievaluasi menggunakan sisa 70% data primer atau 700 *email* sebagai data uji final. Pemilihan proporsi 30% digunakan sebagai desain eksperimen untuk menjaga keseimbangan antara kebutuhan adaptasi domain dan kebutuhan evaluasi model. Pertimbangan pemilihan rasio 30% dijelaskan lebih lanjut melalui eksperimen variasi rasio data adaptasi pada bagian hasil pengujian.

3.1.1. Kerangka Penelitian



Gambar III. 1
Kerangka Penelitian (Metode 1)

Sumber : (Hasil Penelitian, 2026)



Gambar III.2
Kerangka Penelitian Domain Adaptation (Metode 2)

Sumber : (Hasil Penelitian, 2026)

1. Pengumpulan Data (*Data Collection*)

Data yang digunakan dalam penelitian ini terdiri dari dua sumber yang digabungkan untuk mengatasi tantangan *Concept Drift*. Data sekunder (domain sumber) diekstraksi dari *Dataset* publik Kaggle (*Emails.csv*) yang berisi 5.728 pesan *Email* berbahasa Inggris. Sementara itu, data primer (domain target) diekstraksi secara langsung dari kotak masuk (*inbox*) *Email* pribadi peneliti, yang mencakup karakteristik lalu lintas *email* modern seperti notifikasi keamanan dan *One-Time Password* (OTP).

2. Penyeimbangan *Dataset* (*Data Balancing*)

Untuk menghindari bias evaluasi yang disebabkan oleh ketidakseimbangan kelas (*imbalanced data*), data primer dari *Email* pribadi diseimbangkan (*balancing*) secara acak (*random sampling*). Data primer diseimbangkan menjadi tepat 1.000 *Email*, yang terdiri dari 500 *Email* berlabel *spam* (1) dan 500 *Email* berlabel *non-spam* (0). Penyeimbangan ini krusial untuk mencegah terjadinya akurasi semu (*Accuracy Paradox*) pada saat pengujian.

3. Injeksi *Domain Adaptation* dan *Instance Weighting* (Metode 2)

Pada skenario Metode 2, proses penyesuaian model dilakukan untuk menjembatani kesenjangan kosakata (*vocabulary gap*). Proporsi sebesar 30% dari data primer (150 *spam* dan 150 *non-spam*) diekstraksi menggunakan metode *Stratified Sampling* dan disuntikkan ke dalam set pelatihan Kaggle. Karena sampel adaptasi ini kalah jumlah secara cukup besar dibandingkan data Kaggle, teknik *Instance Weighting* (Pembobotan Instans) diimplementasikan. Melalui parameter konfigurasi pelatihan, sampel primer yang disuntikkan diberikan amplifikasi bobot matematis (*sample_weight*) sebesar 8,0. Intervensi ini secara struktural mengarahkan proses

pembelajaran model untuk memprioritaskan fitur kosakata baru di atas pola masa lalu. Sisa 70% data primer (700 *Email*) dipisahkan secara ketat untuk digunakan murni sebagai Data Uji.

Pembobotan ini diaplikasikan secara seragam pada kedua algoritma. Pada *XGBoost*, parameter *sample_weight* diteruskan ke dalam fungsi pelatihan untuk mempengaruhi proses pembentukan pohon keputusan. Sementara pada *Complement Naive Bayes*, parameter yang sama diteruskan ke fungsi *fit()* untuk mempengaruhi perhitungan estimasi probabilitas posterior, sehingga sampel data primer modern memberikan kontribusi delapan kali lipat dibanding sampel historis dalam menentukan distribusi statistik fitur.

4. Pra-Pemrosesan Data (*Text Preprocessing*)

Sebelum diekstraksi menjadi fitur numerik, data teks mentah harus dibersihkan untuk mengurangi dimensi *noise* (Mantha, 2025). Tahapan *Preprocessing* yang dilakukan meliputi:

- a. Case Folding: Mengonversi seluruh teks menjadi huruf kecil (*lowercase*).
- b. Tokenization Khusus: Mengganti elemen spesifik berupa tautan (URL), alamat *Email*, dan angka menjadi token khusus (seperti *urltoken*, *Emailtoken*, dan *numtoken*). Hal ini dilakukan agar model mempelajari struktur manipulatif pesan tanpa menghafal alamat URL atau angka spesifik (Durga Prasad et al., 2026).
- c. Cleansing: Menghapus karakter *non*-alfabetik dan tanda baca yang tidak relevan.

5. Ekstraksi Fitur *Hybrid* Multi-Level

Teks mentah yang telah melalui tahap pra-pemrosesan tidak dapat diproses secara langsung oleh algoritma *Machine Learning*, sehingga memerlukan transformasi

ke dalam representasi ruang vektor matematis (*vector space model*). Penelitian ini mengimplementasikan pendekatan ekstraksi fitur hibrida yang menggabungkan fitur statistik dan fitur heuristik.

- a. Word TF-IDF: Memetakan frekuensi term kata berurutan menggunakan rentang *N-Gram* (1,2) dengan batas maksimal 20.000 fitur, serta menggunakan transformasi logaritmik (*sublinear TF*).
- b. Character TF-IDF: Diimplementasikan dengan rentang *N-Gram* (3,5) dan batas 10.000 fitur metode 1 dan 8.000 fitur metode 2 untuk mendeteksi distorsi ejaan karakter (*intentional typos*) yang menjadi taktik *spam* mutakhir.
- c. Heuristik Kustom: Penambahan matriks yang mengkalkulasi 57 fitur rasio matematis untuk Metode 1 dan 51 fitur rasio matematis untuk Metode 2, termasuk kepadatan kapitalisasi, simbol absolut (! dan \$), *tag* HTML, dan elemen kata sandi. Matriks tipe ini dikonversi secara ketat ke dalam *datatype Float32* untuk kompatibilitas komputasi akselerasi GPU.

6. Pelatihan Algoritma dan Reduksi Dimensi

Selain reduksi dimensi melalui *Chi-Square*, algoritma *Complement Naive Bayes* juga menerapkan strategi pembobotan sampel (*sample weighting*) yang berbeda pada kedua skenario. Pada Metode 1, pembobotan dihitung berdasarkan rasio ketidakseimbangan kelas pada korpus *Kaggle* (sekitar 5.728, yaitu 4.360 *non-spam* dibanding 1.368 *spam*) untuk mencegah model bias terhadap kelas mayoritas. Pada Metode 2, pembobotan dialihfungsikan untuk mendukung *Domain Adaptation*, di mana 300 sampel data primer yang disuntikkan diberikan bobot amplifikasi sebesar 8,0 sama persis dengan strategi yang diterapkan pada *XGBoost*. Perbedaan strategi ini mencerminkan perbedaan tujuan utama: Metode 1 fokus pada penyeimbangan kelas, sedangkan Metode 2 fokus pada penyeimbangan domain.

Hasil dari penyatuan ekstraksi vektor tersebut memproduksi matriks berdimensi sangat tinggi (lebih dari 28.000 fitur). Untuk algoritma *Complement Naive Bayes* (CNB), matriks fitur secara eksplisit dipaksa melewati proses *Feature Selection* otomatis menggunakan modul *SelectKBest*. Proses ini mengaplikasikan uji statistik *Chi-Square* (X^2) untuk mereduksi *noise* dan hanya meloloskan 12.000 fitur yang memiliki tingkat determinasi klasifikasi tertinggi.

Algoritma *Complement Naive Bayes* dikonfigurasi dengan parameter pemulusan (*smoothing parameter*) α yang berbeda pada kedua skenario. Pada Metode 1, nilai α ditetapkan sebesar 0,05 untuk meminimalkan efek pemulusan dan mempertahankan sensitivitas terhadap fitur diskriminatif pada data historis yang berdimensi tinggi. Pada Metode 2, nilai α dinaikkan menjadi 0,1 untuk memberikan stabilitas numerik tambahan pada estimasi probabilitas, mengingat adanya injeksi sampel data modern yang memiliki distribusi kosakata berbeda dari korpus utama. Nilai α yang lebih besar mencegah terjadinya probabilitas nol (*zero probability*) pada token baru yang hanya muncul di sampel adaptasi.

Sebaliknya, arsitektur *Extreme Gradient Boosting* (*XGBoost*) memproses keseluruhan 28.000+ dimensi tersebut secara langsung. *XGBoost* mengandalkan utilitas komputasi paralel pada *Graphical Processing Unit* (GPU) berbasis CUDA, serta menerapkan otomatisasi regularisasi di dalam arsitektur internalnya (parameter *gamma*, *reg_alpha*, dan *reg_lambda*) untuk membatasi kompleksitas model dan mencegah *overfitting*.

7. Optimasi Ambang Batas (*Dynamic Threshold Tuning*) dan Evaluasi

Sebagai tahap penyeimbangan klasifikasi sebelum evaluasi final, penelitian ini tidak menggunakan ambang batas tebakan probabilitas bawaan mesin (0,5). Sistem mengekstraksi nilai probabilitas prediksi murni (*predict_proba*) pada

data validasi terpisah, kemudian melakukan perulangan iteratif dari rentang 0,10 - 0,95 untuk Metode 1 dan 0,15 - 0,85 untuk Metode 2. Titik probabilitas pemisah yang menghasilkan perolehan *F1-score* tertinggi pada iterasi tersebut dikunci sebagai ambang batas terbaik (*Best Threshold*). Nilai *threshold* dinamis inilah yang kemudian digunakan secara mutlak untuk memprediksi kelas Data Uji (*Test Set*) untuk memperoleh laporan akhir *Confusion matrix* (Akurasi, Presisi, *Recall*, dan *F1-score*) yang sesungguhnya.

3.2. Metode Pengolahan dan Analisis Data

3.2.1. Metode Pengolahan Data

Proses pengolahan data mentah menjadi informasi numerik yang siap diproses oleh algoritma klasifikasi dilakukan melalui serangkaian tahapan rekayasa fitur (*feature engineering*) menggunakan bahasa pemrograman Python. Tahapan pengolahan data dibagi menjadi tiga komponen utama:

1. Ekstraksi Fitur (*Feature Engineering & TF-IDF*)

Pengolahan data dilakukan melalui dua pendekatan ekstraksi fitur yang digabungkan (*hstack*):

- a. Ekstraksi Fitur Heuristik (Manual): Dilakukan ekstraksi sebanyak 57 fitur heuristik kustom tambahan untuk metode 1, dan 51 fitur heuristik kustom tambahan untuk metode 2 yang didasarkan pada karakteristik anatomi *email* modern. Fitur ini mencakup kalkulasi rasio panjang teks, kepadatan huruf kapital, jumlah tanda baca spesifik (! dan \$), frekuensi kemunculan *placeholder* (seperti *url token* dan *Email token*), serta deteksi kata kunci manipulatif. Penambahan fitur ini bertujuan untuk memperkuat model dalam

mengenali pola rekayasa sosial yang sering kali tidak tertangkap hanya melalui frekuensi kata (Tian et al., 2025).

- b. Vektorisasi TF-IDF *Hybrid*: Korpus teks diubah menjadi ruang vektor menggunakan metode *Term Frequency-Inverse Document Frequency* (TF-IDF). Proses ini dilakukan pada dua level, yaitu tingkat kata (*Word N-Grams*) untuk menangkap konteks frasa, dan tingkat karakter (*Character N-Grams*) untuk mengatasi manipulasi ejaan. Hasil dari kedua vektor ini digabungkan secara horizontal (*hstack*) bersama fitur heuristik untuk membentuk *Sparse matrix* berdimensi tinggi (B.Sruthi et al., 2026).

2. Pustaka (*Library*) Pemrosesan

Integrasi Pustaka Pemrosesan: Seluruh proses manipulasi matriks dan aljabar linear dikelola menggunakan pustaka Pandas dan NumPy. Tahap *Preprocessing* dan seleksi fitur *Chi-Square* dieksekusi menggunakan modul Scikit-Learn, sedangkan proses pelatihan ansambel berbasis GPU diproses menggunakan pustaka *XGBoost Library* (Jáñez-Martino et al., 2025).

3.2.2. Analisis Data

Data hasil pengujian dari kedua skenario eksperimen (Metode 1 dan Metode 2) dianalisis secara komparatif untuk mengukur pengaruh *Domain Adaptation* terhadap kinerja algoritma. Analisis dilakukan tidak hanya berdasarkan angka akurasi global, melainkan melalui pembedahan distribusi prediksi menggunakan instrumen *Confusion matrix*.

Confusion matrix memetakan hasil prediksi model ke dalam empat indikator utama: *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False Negative* (FN) (Lukito & Wahyuningrum, 2025).

Berdasarkan indikator tersebut, analisis performa dikalkulasi menggunakan metrik sebagai berikut:

- a. Akurasi: Mengukur efektivitas total model dalam mengklasifikasikan seluruh data uji secara benar.
- b. Presisi: Mengukur tingkat ketepatan model dalam menebak kelas *spam*. Metrik ini menjadi fokus utama analisis untuk memastikan model tidak melakukan kesalahan fatal berupa salah klasifikasi *email* penting menjadi *spam* (*False Positive*).
- c. *Recall* (Sensitivitas): Mengukur kemampuan model dalam menangkap keseluruhan pesan *spam* yang sebenarnya masuk ke dalam sistem.
- d. *F1-score*: Digunakan sebagai parameter keseimbangan paling objektif untuk mengevaluasi performa model pada data lintas domain yang mengalami *Concept Drift*.

Analisis data ditutup dengan membandingkan hasil akhir dari kedua metode. Kemerosotan performa pada Metode 1 akan dianalisis sebagai bukti adanya pergeseran konsep data, sementara pemulihan nilai akurasi pada Metode 2 akan dianalisis sebagai bukti efektivitas penggunaan data primer dan teknik pembobotan instans dalam menjaga relevansi sistem keamanan *email* (Akpan et al., 2026).

BAB IV

HASIL DAN PEMBAHASAN

4.1. Hasil Penelitian

Berdasarkan parameter metodologi komputasional yang telah ditetapkan, bagian ini menguraikan hasil eksekusi tahapan algoritma secara sekuensial. Pemrosesan data teks hingga pelatihan model dieksekusi secara terpusat menggunakan bahasa pemrograman Python.

4.1.1. Hasil Pengumpulan dan Distribusi Data Lintas Domain

Pemrosesan matriks diawali dengan pendistribusian proporsi data untuk dua skenario eksperimen secara ketat guna mendemonstrasikan dampak *Concept Drift*:

1. Skenario Metode 1 (Model Murni): Data latih diekstraksi 100% dari *Dataset* historis Kaggle dengan jumlah 5.728 *Email* (4.360 *non-spam* dan 1.368 *spam*). Pengujian (*testing*) dilakukan terhadap 1.000 sampel murni dari *email* personal yang telah diseimbangkan secara ketat menjadi 500 *non-spam* dan 500 *spam*.

Tabel IV. 1
Skenario Metode 1

	Sekunder	Primer
<i>Spam</i>	1.368 <i>Email</i>	500 <i>Email</i>
<i>Non-Spam</i>	4.360 <i>Email</i>	500 <i>Email</i>
jumlah	5.728 <i>Email</i>	1.000 <i>Email</i>

Sumber : (Hasil Penelitian 2026)

2. Skenario Metode 2 (*Domain Adaptation*): Sebanyak 30% dari populasi data personal (sebanyak 300 *Email* terdiri dari 150 *email spam* dan 150 *email non-spam*) ditambahkan secara paksa ke dalam korpus latih Kaggle. Hal ini menjadikan

total himpunan data latih bertambah menjadi 6.028 terdiri dari 1518 *email spam* dan 4510 *email non-spam*. Sisa 70% dari data personal (berjumlah 700 *Email*, terdiri dari 350 *non-spam* dan 350 *spam*) dipisahkan secara eksklusif sebagai Data Uji final.

Tabel IV. 2
Skenario Metode 2

	Data Train Sekunder	Data Test Primer
<i>Spam</i>	1.368 <i>Email</i>	500 <i>Email</i>
<i>Non-Spam</i>	4.360 <i>Email</i>	500 <i>Email</i>
<i>Adaptation</i>	+ 30% Data Primer	- 30% untuk <i>Domain Adaptation</i>
jumlah	6.028 <i>Email</i>	700 <i>Email</i>

Sumber : (Hasil Penelitian, 2026)

4.1.2. Hasil Rekayasa Fitur dan Pembentukan Matriks *Hybrid*

Teks *email* mentah tidak dapat dikalkulasi secara aljabar sehingga harus melewati tahap normalisasi dan ekstraksi. Fitur teks dikonversi melalui pembobotan TF-IDF tingkat kata dan tingkat karakter (rentang 3-5 gram), yang kemudian digabungkan secara horizontal (*hstack*) dengan matriks 57 fitur heuristik kustom untuk metode 1, dan 51 fitur heuristik kustom untuk metode 2 (seperti deteksi kepadatan kapitalisasi dan anomali tanda baca).

Hasil dari vektorisasi matriks ini memproduksi luaran dimensi yang berbeda:

1. Pada Metode 1, proses peleburan fitur memproduksi *Sparse matrix* berdimensi sangat padat sebanyak 30.057 fitur.
2. Pada Metode 2, menggunakan konfigurasi parameter ekstraksi fitur yang berbeda dengan Metode 1, di mana *max_features* karakter TF-IDF dibatasi pada 8.000 (dari

10.000) dan jumlah kata kunci *spam* yang lebih ringkas, sehingga total dimensi fitur menjadi 28.051 fitur (dikonversi ke format *datatype Float32* khusus untuk GPU). Matriks berdimensi puluhan ribu inilah yang kemudian dimasukkan ke dalam algoritma pelatihan.

4.1.3. Implementasi Seleksi Fitur dan *Instance Weighting*

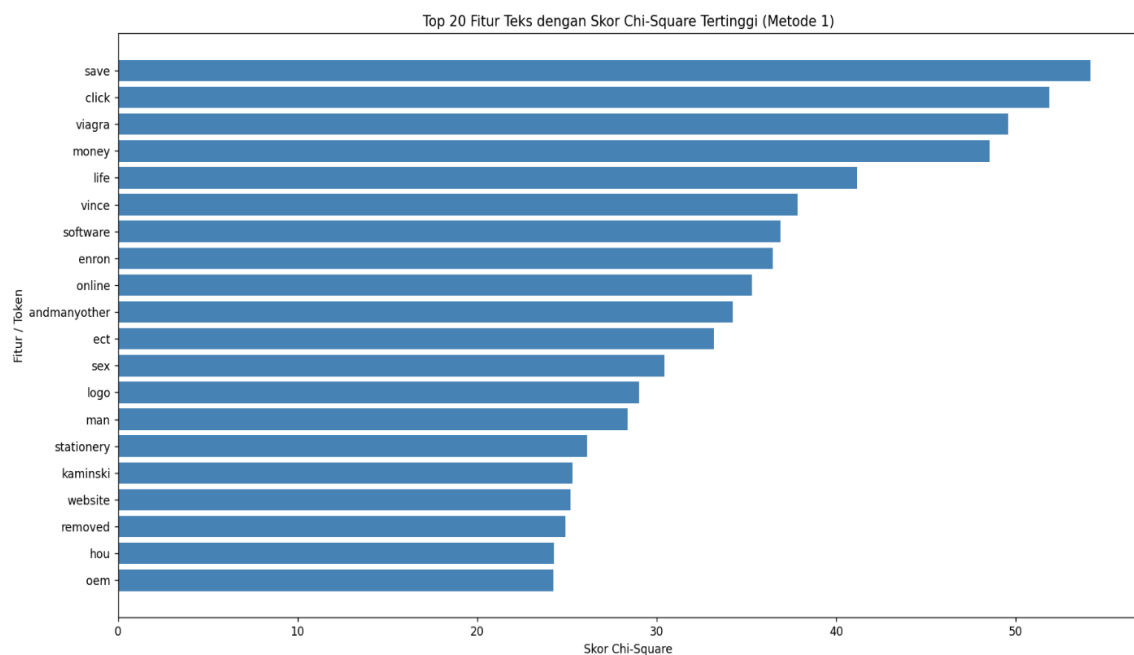
Besarnya dimensi matriks memaksa penerapan intervensi yang berbeda pada algoritma. Untuk algoritma probabilistik *Complement Naive Bayes* (CNB), seluruh beban matriks dipaksa melewati modul *SelectKBest* yang menggunakan uji statistik *Chi-Square* (X^2). Hasilnya, sistem membuang belasan ribu *noise* yang tidak independen dan secara aktual hanya mempertahankan 12.000 fitur paling prediktif agar model CNB tidak mengalami penurunan performa akibat jumlah fitur yang terlalu besar.

Pada penelitian ini, penggunaan fitur pada kedua algoritma tidak sepenuhnya sama. *Complement Naive Bayes* menggunakan fitur *Word TF-IDF* yang telah diseleksi menggunakan metode *Chi-Square*, karena algoritma probabilistik ini lebih sensitif terhadap jumlah fitur yang terlalu besar. Sementara itu, *XGBoost* menggunakan matriks fitur *hybrid* yang terdiri dari *Word TF-IDF*, *Character TF-IDF*, dan fitur heuristik. Perbedaan ini dilakukan karena *XGBoost* lebih mampu menangani fitur berdimensi tinggi dan hubungan antarfitur yang lebih kompleks dibandingkan *Complement Naive Bayes*.

Khusus pada Skenario Metode 2, implementasi *Instance Weighting* dieksekusi melalui pemberian bobot lebih tinggi (*sample_weight*). 300 sampel data personal yang disuntikkan diberikan bobot sebesar 8,0x. Amplifikasi ini mempengaruhi proses pembelajaran model algoritma agar memberikan atensi dan

penalti yang jauh lebih berat pada kesalahan klasifikasi kosakata *email* modern dibandingkan kesalahan pada korpus usang.

Pada Metode 1, fitur Word TF-IDF diseleksi menggunakan modul *SelectKBest* dengan uji statistik *Chi-Square*, di mana hanya fitur dengan skor tertinggi yang dipertahankan untuk membangun model. Visualisasi kontribusi fitur pada pendekatan tanpa adaptasi ini dapat dilihat pada Gambar IV.1.



Gambar IV. 1

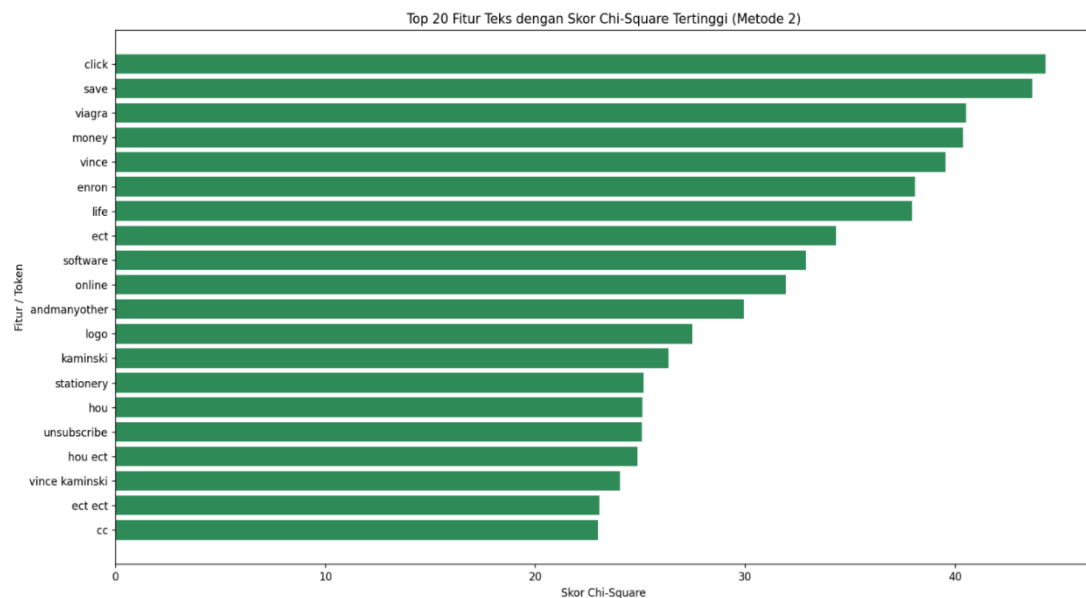
Grafik Top 20 Fitur *Chi-Square* (Metode 1)

Sumber : (Hasil Penelitian, 2026)

Berbeda dengan skenario sebelumnya, pada Metode 2 sampel data primer modern yang disuntikkan diberikan bobot amplifikasi absolut sebesar 8,0. Perubahan prioritas fitur akibat pembobotan ini divisualisasikan pada Gambar IV.2.

Grafik pada Gambar IV.2 menunjukkan bahwa setelah intervensi *Domain Adaptation*, hierarki fitur penting secara drastis mulai mencerminkan pengaruh data primer modern. Token-token yang relevan dengan lalu lintas *email* aktual saat ini mulai mendapat prioritas atensi dari algoritma. Pergeseran skor *Chi-Square* ini menunjukkan bahwa pembobotan instans secara efektif memaksa model untuk beradaptasi dengan

lingkungan target (*target domain*). Grafik ini secara spesifik merupakan metrik dukungan pada tahap rekayasa dan seleksi fitur, bukan merepresentasikan hasil akurasi evaluasi akhir.



Gambar IV. 2
Grafik Top 20 Fitur *Chi-Square* (Metode 2)

Sumber : (Hasil Penelitian, 2026)

4.1.4. Hasil Pelatihan Model dan Optimasi *Threshold*

Pelatihan algoritma ansambel *Extreme Gradient Boosting* (*XGBoost*) dieksekusi langsung pada matriks utuh (tanpa seleksi *SelectKBest*) dengan memanfaatkan akselerasi *hardware* GPU (CUDA RTX 3050) dan arsitektur *tree-method histogram*. Eksekusi komputasional pada *Metode 1* menghasilkan titik penghentian otomatis (*early stopping*) pada pohon ke-828. Sedangkan pada *Metode 2*, pemberian bobot sebesar 8,0 pada sampel data modern membuat algoritma *XGBoost* memberikan perhatian lebih besar terhadap pola dari data target, sehingga proses pembelajaran berlangsung lebih panjang hingga mencapai iterasi terbaik pada pohon ke-1708. Jumlah iterasi yang lebih besar ini mengindikasikan bahwa algoritma

memerlukan proses pembelajaran yang lebih intensif untuk mengakomodasi distribusi fitur dari dua domain yang berbeda secara simultan.

Setelah model selesai dilatih, ambang batas probabilitas (0,5) diabaikan. Keputusan akhir dieksekusi dengan pencarian *Dynamic Threshold Tuning* pada probabilitas data validasi. Proses ini mengunci nilai titik potong terbaik (*Best Threshold*) yang menghasilkan *F1-score* harmonis tertinggi sebelum diujikan pada data murni, dengan hasil sebagai berikut:

1. Metode 1: Ambang batas optimal untuk *Complement Naive Bayes* terkunci di 0,42 sedangkan *XGBoost* di 0,73.
2. Metode 2: Ambang batas optimal untuk *Complement Naive Bayes* terkunci di 0,51 sedangkan *XGBoost* di 0,64.

Seluruh arsitektur model beserta parameter probabilitas optimal inilah yang kemudian divalidasi kelemahannya melalui konfrontasi matriks kebingungan (*Confusion matrix*) yang sesungguhnya pada tahap Pengujian.

4.2. Hasil Pengujian

Bagian ini menyajikan analisis mendalam terhadap performa model klasifikasi *Email spam* setelah melewati tahap pengujian menggunakan data uji murni yang independen. Pengujian ini ditujukan untuk memetakan kapasitas generalisasi model, membedah distribusi kesalahan prediksi melalui instrumen *Confusion matrix*, serta membandingkan efisiensi Metode 1 (*Tanpa Adaptasi*) dan Metode 2 (*Domain Adaptation*) secara empiris guna menjawab rumusan masalah penelitian.

4.2.1. Hasil Pengujian Metode 1 (Tanpa Adaptasi)

Pada skenario Metode 1, model yang secara eksklusif dilatih menggunakan *Dataset* historis (Kaggle) langsung diuji kemampuannya untuk

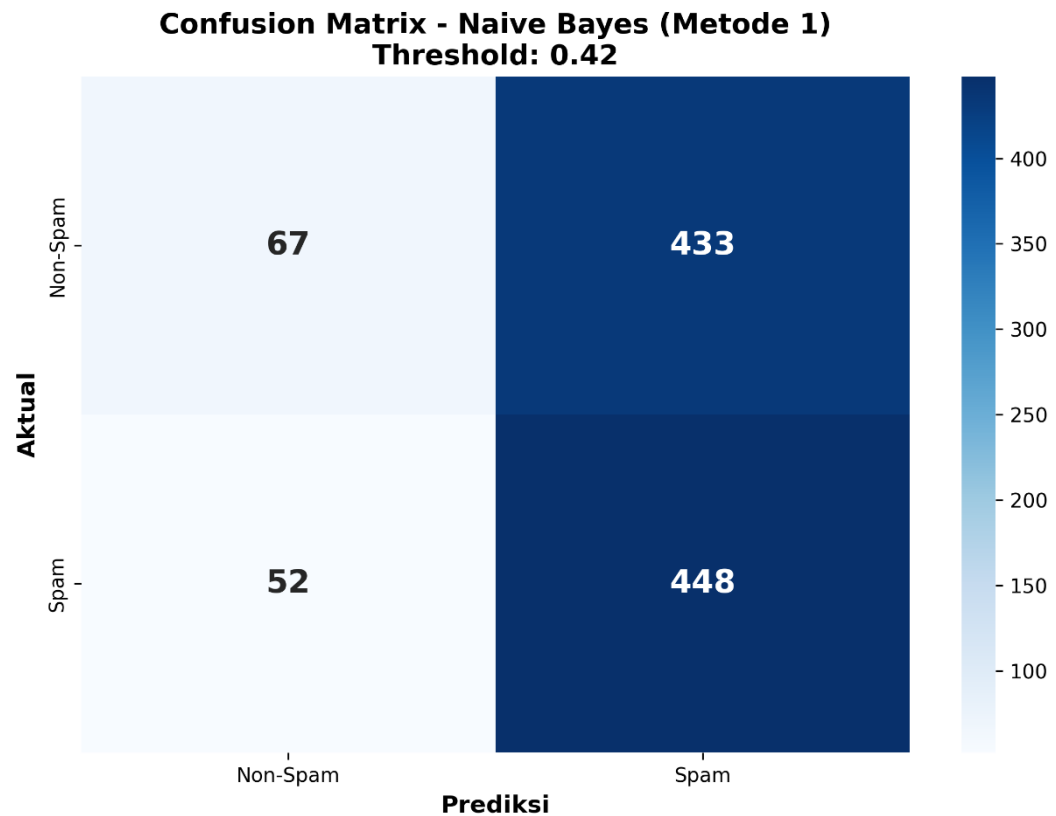
mengklasifikasikan *Email* personal modern. Pengujian ini secara sengaja dirancang untuk mengukur kemampuan generalisasi model algoritma akibat *Concept Drift*. Ringkasan hasil evaluasi performa dari kedua algoritma disajikan pada Tabel IV.3 berikut:

Tabel IV. 3
Hasil Pengujian Metode 1 (Tanpa Adaptasi)

Model	Akurasi	Presisi	<i>Recall</i>	<i>F1-score</i>
<i>Complement Naive Bayes</i>	51,50%	53,58%	51,50%	43,26%
<i>XGBoost</i>	48,20%	48,15%	48,20%	47,87%

Sumber : (Hasil Penelitian, 2026)

Berdasarkan Tabel IV.3, terlihat jelas bahwa model klasifikasi berbasis data historis menunjukkan performa rendah dalam mengklasifikasikan *Email* modern. Untuk membedah letak kesalahan operasional dari sistem, distribusi prediksi *Confusion matrix* dari masing-masing algoritma divisualisasikan pada Gambar IV.3 dan Gambar IV.4.



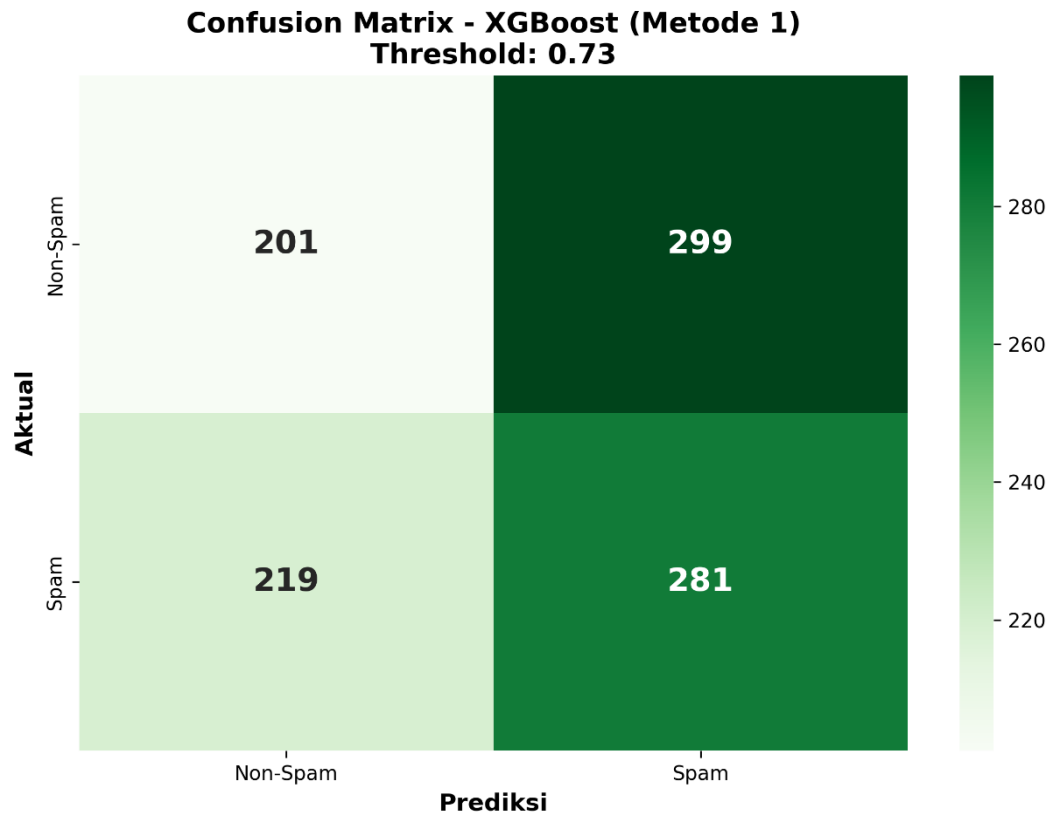
Gambar IV. 3 *Confusion matrix Complement Naive Bayes* Metode 1

Sumber : (Hasil Penelitian, 2026)

Berdasarkan Gambar IV.3, algoritma *Complement Naive Bayes* menunjukkan kecenderungan kesalahan prediksi yang tinggi. Algoritma ini secara keliru banyak mengklasifikasikan dan mengklasifikasikan pesan sah (*non-spam*) menjadi pesan sampah (*spam*) dalam jumlah yang besar, di mana nilai *False Positive* mencapai 433 insiden. Hal ini menunjukkan bahwa tanpa penyesuaian domain, model belum mampu mengenali konteks *Email* modern secara baik dan menurunkan keandalan model pada data uji modern di kotak masuk pengguna.

Pada Metode 1, *Complement Naive Bayes* menghasilkan jumlah *false positive* yang tinggi, yaitu banyak *Email* sah (*non-spam*) yang salah diklasifikasikan sebagai *spam*. Kondisi ini menunjukkan bahwa model yang hanya dilatih menggunakan *Dataset* historis belum mampu membedakan karakteristik *Email* sah modern secara baik. Kesalahan ini cukup penting dalam sistem deteksi *spam* karena

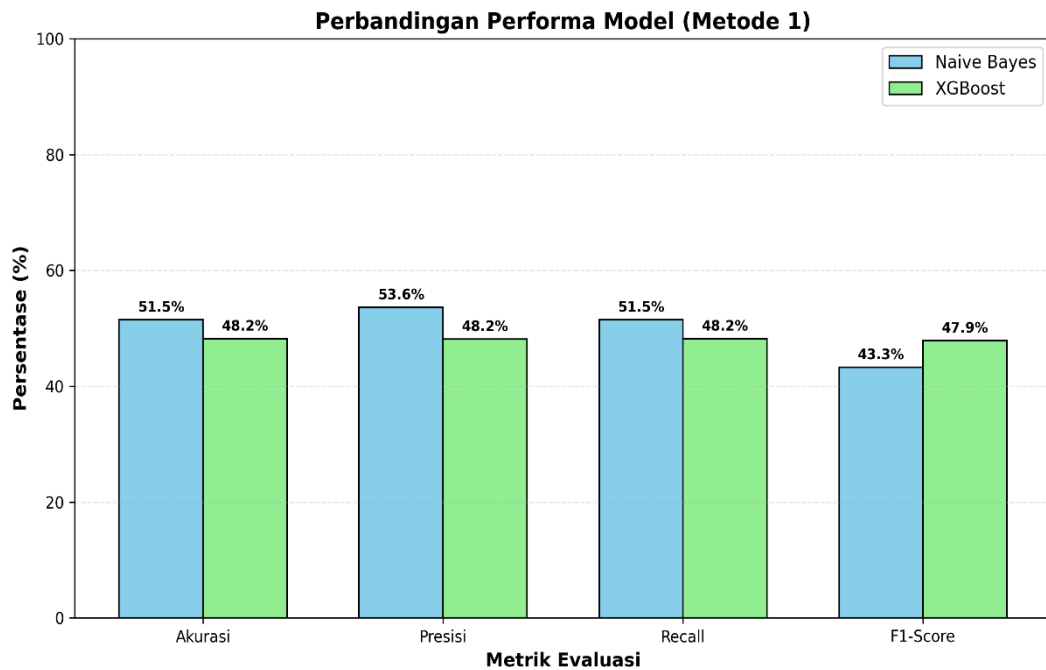
Email sah yang salah masuk ke kategori *spam* dapat menyebabkan informasi penting tidak terbaca oleh pengguna.



Gambar IV. 4 *Confusion matrix XGBoost Metode 1*

Sumber : (Hasil Penelitian, 2026)

Analisis terhadap Gambar IV.4 juga memperlihatkan ketidakstabilan klasifikasi pada *XGBoost*. Meskipun memiliki arsitektur *ensemble* yang kompleks, algoritma ini hanya mampu mencapai akurasi sebesar 48,20%. Ketidakmampuan menyaring target ini menunjukkan bahwa kecanggihan algoritma secanggih apa pun akan tetap lumpuh jika distribusi wawasan (data latih) dan lingkungan operasional (data uji) mengalami perbedaan yang drastis.



Gambar IV. 5 Perbandingan Metrik Evaluasi Metode 1

Sumber : (Hasil Penelitian, 2026)

Secara visual, Gambar IV.5 merangkum keterpurukan performa kedua algoritma yang hanya terhenti di ambang batas 50%. Secara komputasional, angka ini tidak lebih baik daripada tebakan acak. Lemahnya performa dalam seluruh metrik evaluasi ini adalah validasi awal yang kuat atas terjadinya fenomena perbedaan distribusi data (*domain mismatch*).

4.2.2. Hasil Pengujian Metode 2 (*Domain Adaptation*)

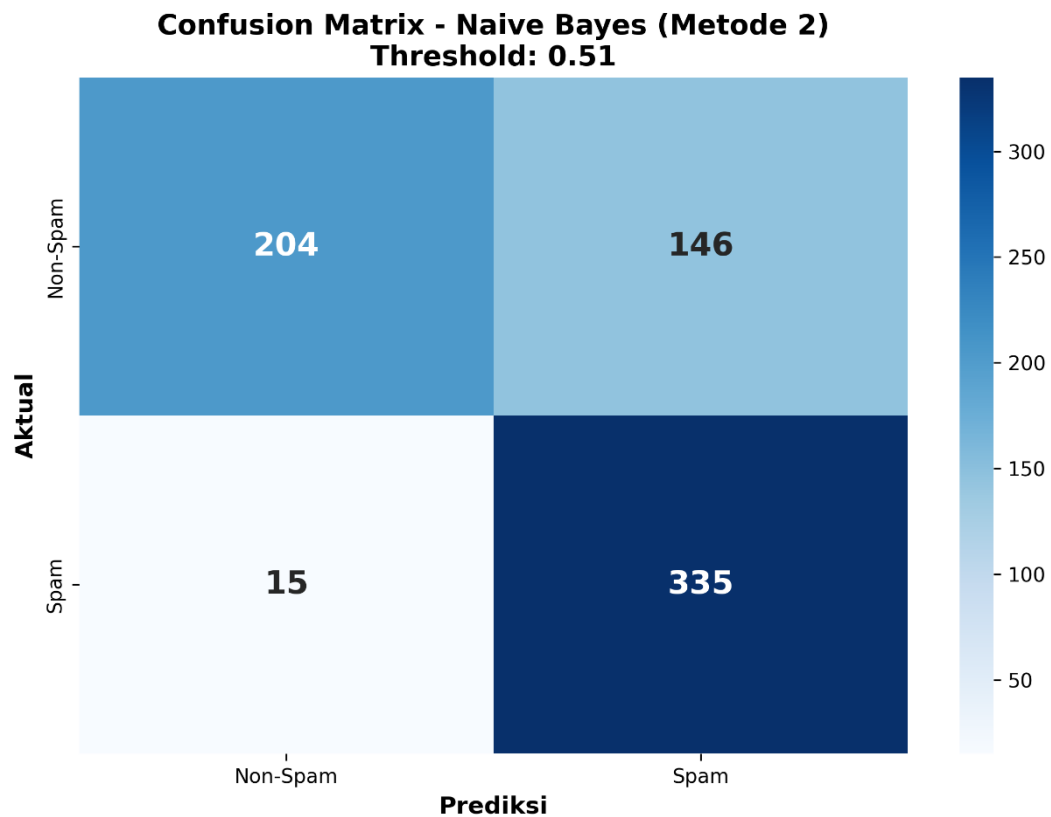
Pada skenario Metode 2, penerapan *Domain Adaptation* diaplikasikan melalui injeksi sampel aktual dan amplifikasi bobot (*Instance Weighting*) sebesar 8,0. Pengujian ini difokuskan pada pengukuran tingkat peningkatan performa model klasifikasi. Hasil evaluasi disajikan pada Tabel IV.4 berikut:

Tabel IV. 4 Hasil Pengujian Metode 2 (*Domain Adaptation*)

Model	Akurasi	Presisi	Recall	F1-score
<i>Complement Naive Bayes</i>	77,00%	81,40%	77,00%	76,17%
<i>XGBoost</i>	93,00%	93,08%	93,00%	93,00%

Sumber : (Hasil Penelitian, 2026)

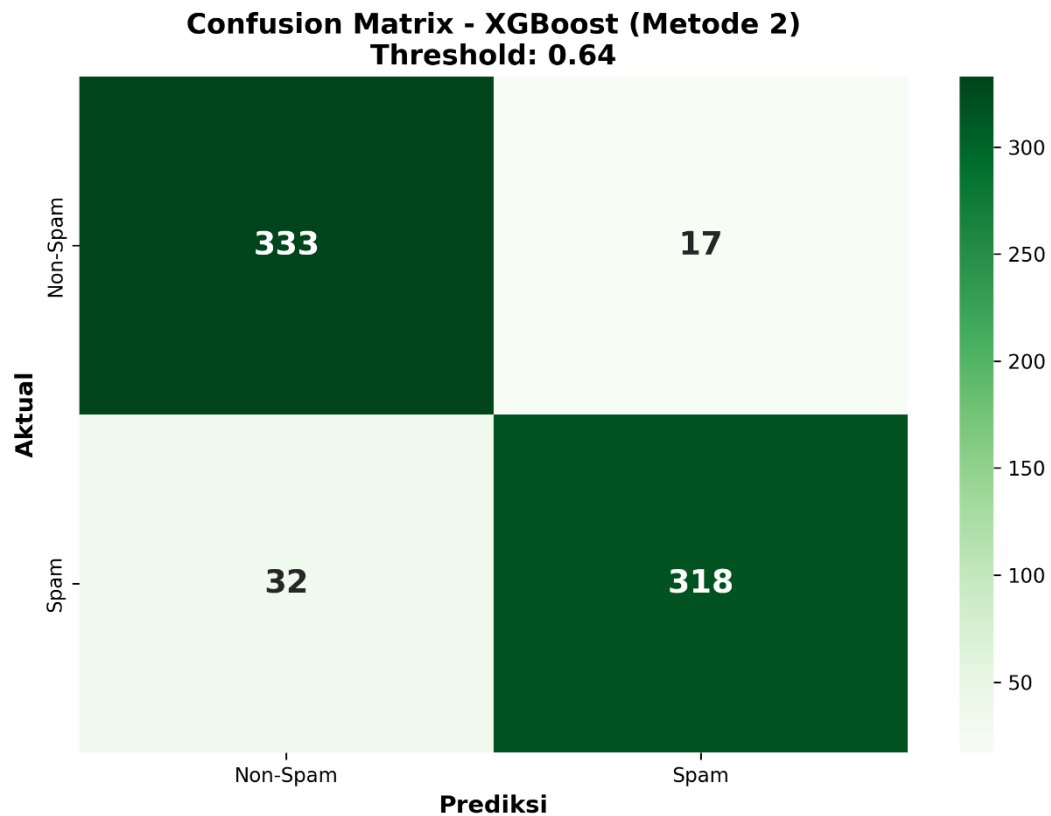
Pada Tabel IV.4, dapat dipastikan bahwa kedua model mengalami lonjakan kualitas yang signifikan. Pembedaan mengenai ketajaman model dalam menebak kategori secara aktual dijabarkan dalam *Confusion matrix* pada Gambar IV.6 dan Gambar IV.7.



Gambar IV. 6 *Confusion matrix Complement Naive Bayes Metode 2*

Sumber : (Hasil Penelitian, 2026)

Gambar IV.6 menunjukkan bahwa *Complement Naive Bayes* mengalami pemulihan performa yang cukup rasional. Kemampuannya mendeteksi *email spam* yang sebenarnya (*True Positive*) menjadi sangat tajam, menembus angka 335 dari 350 pesan aktual. Walaupun demikian, model ini masih rentan akibat nilai *False Positive* yang cukup besar (146 *email* sah ditandai sebagai *spam*). Dengan kata lain, kinerja *Complement Naive Bayes* mengalami perbaikan, tetapi agresivitas probabilitasnya masih membahayakan *Email* penting pengguna.



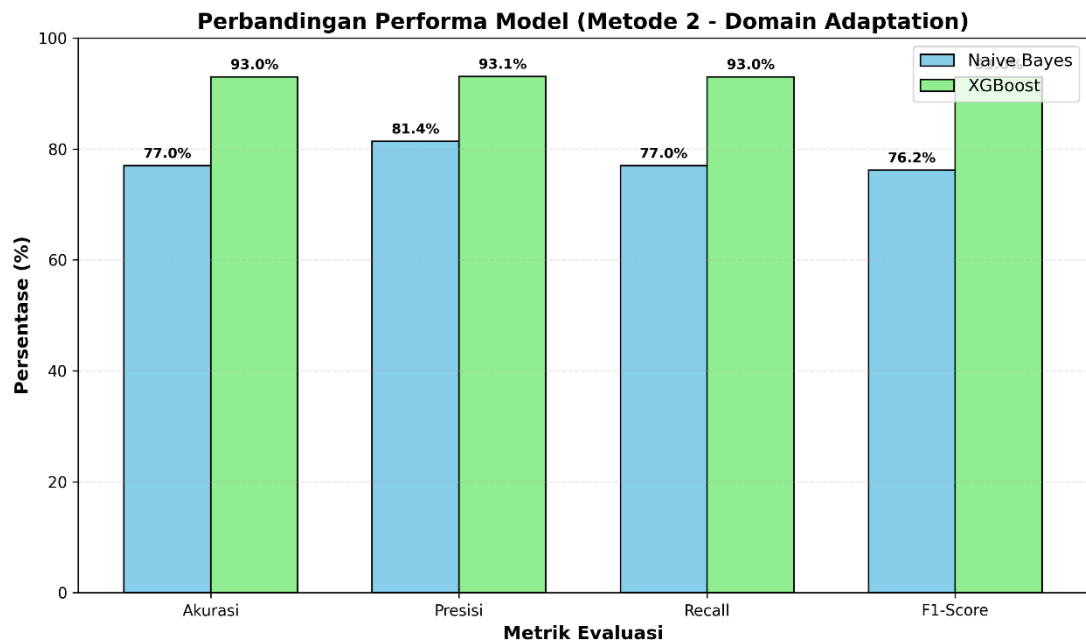
Gambar IV. 7 *Confusion matrix XGBoost Metode 2*

Sumber : (Hasil Penelitian, 2026)

Sebaliknya, pemetaan pada Gambar IV.7 memvalidasi superioritas *XGBoost* dalam membedah ruang dimensi fitur silang-domain. Algoritma *XGBoost* berhasil menata ulang batas keputusannya secara optimal dengan menekan *False Positive* ke titik sangat minimal (hanya 17 *Email* sah yang menjadi korban kekeliruan). Rasio minimalisasi *False Positive* ini merupakan prasyarat krusial agar pengguna tidak kehilangan *Email* notifikasi atau kata sandi harian mereka akibat tersaring sistem keamanan.

Berdasarkan *Confusion matrix* pada *XGBoost* Metode 2, jumlah *false positive* yang dihasilkan hanya sebanyak 17 *Email*. Nilai ini menunjukkan bahwa model relatif mampu menjaga *Email* sah agar tidak salah diklasifikasikan sebagai *spam*. Dalam konteks klasifikasi *spam*, *false positive* menjadi salah satu kesalahan yang perlu diperhatikan karena dapat menyebabkan *Email* penting masuk ke kategori

spam. Oleh karena itu, rendahnya jumlah *false positive* pada *XGBoost* Metode 2 menunjukkan bahwa model tidak hanya memiliki akurasi tinggi, tetapi juga lebih aman dalam mempertahankan *Email* sah pada kelas yang benar.



Gambar IV. 8 Perbandingan Metrik Evaluasi Metode 2

Sumber : (Hasil Penelitian, 2026)

Gambar IV.8 memvisualisasikan dominasi *XGBoost* yang memperoleh nilai tertinggi pada seluruh metrik evaluasi, dengan skor rata-rata di kisaran 93,00%. peningkatan performa yang besar dibandingkan *baseline* murni ini memperkuat kesimpulan bahwa penyesuaian distribusi data (*Domain Adaptation*) adalah pendekatan yang berperan penting dalam klasifikasi masa kini.

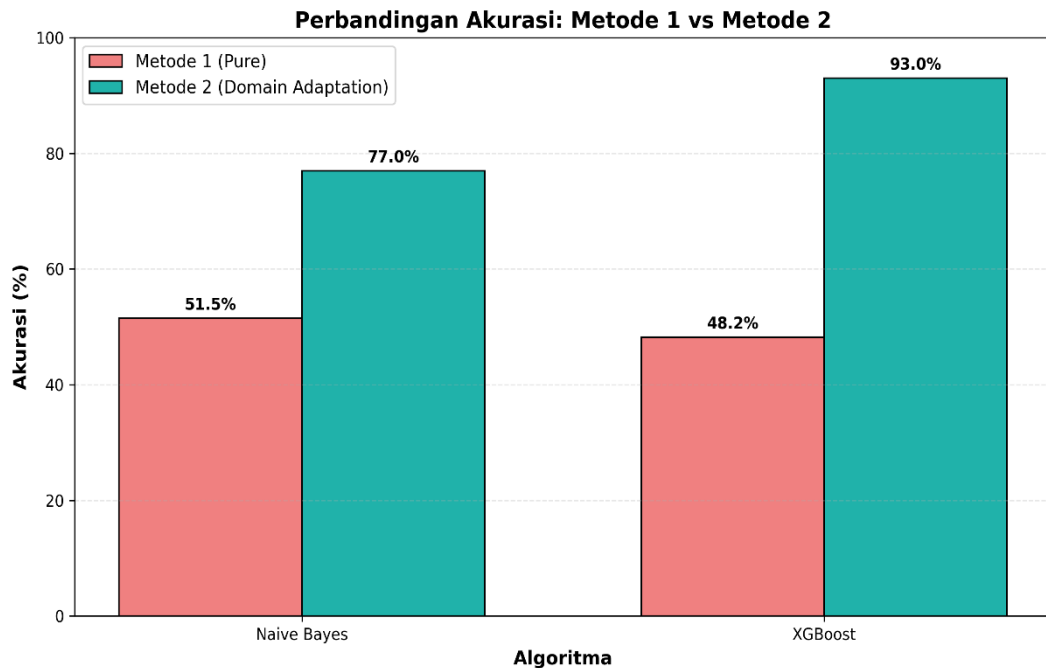
4.2.3. Perbandingan Performa Metode 1 dan Metode 2

Pada bagian ini merupakan perbandingan langsung guna menemukan jawaban komparatif atas rumusan masalah penelitian. Matriks peningkatan selisih performa model sebelum dan sesudah diterapkan adaptasi domain disajikan dalam Tabel IV.5.

Tabel IV. 5 Perbandingan Akurasi Metode 1 dan Metode 2

Algoritma	Metode 1	Metode 2	Peningkatan
<i>Complement Naive Bayes</i>	51,50%	77,00%	+25,50%
<i>XGBoost</i>	48,20%	93,00%	+44,80%

Sumber : (Hasil Penelitian, 2026)



Gambar IV. 9 Perbandingan Akurasi Metode 1 dan Metode 2

Sumber : (Hasil Penelitian, 2026)

Sesuai dengan grafik pada Gambar IV.9, dapat dikonfirmasi bahwa terjadi peningkatan akurasi yang besar. performa rendah yang tersaji di Metode 1 menunjukkan secara eksplisit bahwa model yang hanya dilatih pada data historis belum mampu melakukan generalisasi dengan baik dalam mengenali pola *Email* modern. Sebaliknya, saat diberikan penambahan sebagian data primer dari data aktual sebesar 30%, peningkatan akurasi pun terbentuk. Kenaikan nilai puncak berhasil diperoleh algoritma *XGBoost* dengan peningkatan akurasi tertinggi sebesar +44,80%. Pembuktian ini menunjukkan bahwa *Domain Adaptation* berperan penting sebagai strategi komputasi yang sangat penting dalam mengatasi kesenjangan domain.

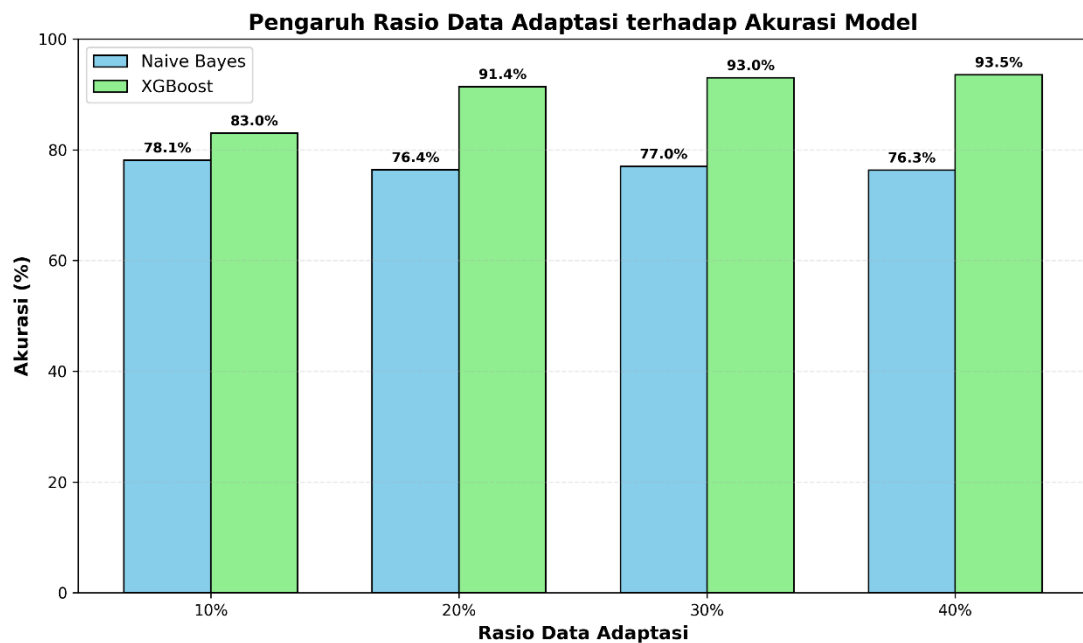
4.2.4. Analisis Tambahan terhadap Rasio Data Adaptasi

Tabel IV. 6 Hasil Eksperimen Variasi Rasio Data Adaptasi

Rasio Adaptasi	Data Adaptasi	Data Uji	NB Akurasi	NB <i>F1-score</i>	XGB Akurasi	XGB <i>F1-score</i>	NB FP	NB FN	XGB FP	XGB FN
10%	100	900	78,11%	77,77%	83,00%	82,95%	154	43	100	53
20%	200	800	76,38%	75,55%	91,38%	91,37%	168	21	25	44
30%	300	700	77,00%	76,17%	93,00%	93,00%	146	15	17	32
40%	400	600	76,33%	75,31%	93,50%	93,50%	132	10	20	19

Sumber : (Hasil Penelitian, 2026)

Selain pengujian utama pada Metode 1 dan Metode 2, penelitian ini juga melakukan eksperimen tambahan untuk melihat pengaruh variasi rasio data adaptasi terhadap performa model. Eksperimen dilakukan dengan membandingkan rasio data adaptasi sebesar 10%, 20%, 30%, dan 40% dari total data primer. Tujuan dari eksperimen ini adalah untuk mengevaluasi apakah penambahan jumlah data target selalu memberikan peningkatan performa yang sebanding dengan berkurangnya jumlah data uji final.



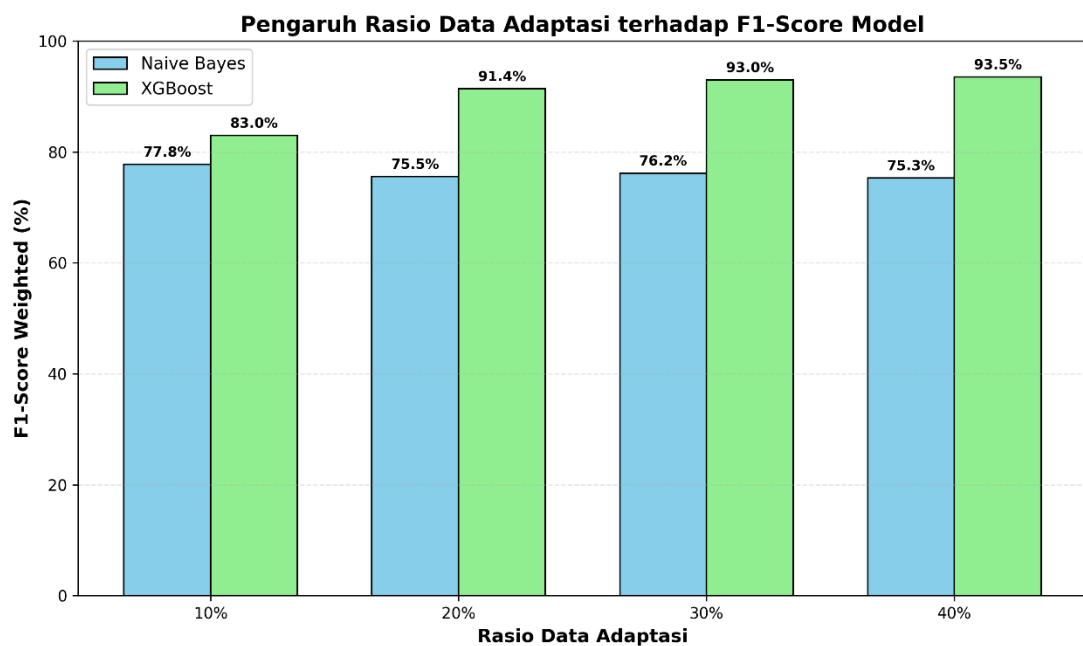
Gambar IV. 10 Eksperimen Rasio Akurasi

Sumber : (Hasil Penelitian, 2026)

Berdasarkan hasil eksperimen, performa *Complement Naive Bayes* relatif stabil pada seluruh rasio adaptasi, yaitu berada pada kisaran 76% hingga 78%. Akurasi

tertinggi *Complement Naive Bayes* memang diperoleh pada rasio 10% sebesar 78,11%, tetapi selisihnya dengan rasio 30% tidak terlalu besar. Hal ini menunjukkan bahwa *Complement Naive Bayes* tidak terlalu sensitif terhadap perubahan jumlah data adaptasi dan cenderung mengalami plateau pada skenario *Domain Adaptation*.

Berbeda dengan *Complement Naive Bayes*, *XGBoost* menunjukkan peningkatan performa yang lebih jelas ketika jumlah data adaptasi ditambah. Pada rasio 10%, *XGBoost* memperoleh akurasi sebesar 83,00%. Nilai tersebut meningkat menjadi 91,38% pada rasio 20%, kemudian naik lagi menjadi 93,00% pada rasio 30%. Pada rasio 40%, akurasi *XGBoost* meningkat menjadi 93,50%, tetapi peningkatannya hanya sebesar 0,50 poin persentase dibandingkan rasio 30%. Kenaikan yang kecil ini menunjukkan adanya kecenderungan diminishing return, yaitu tambahan data adaptasi setelah rasio 30% tidak lagi memberikan peningkatan performa yang besar.

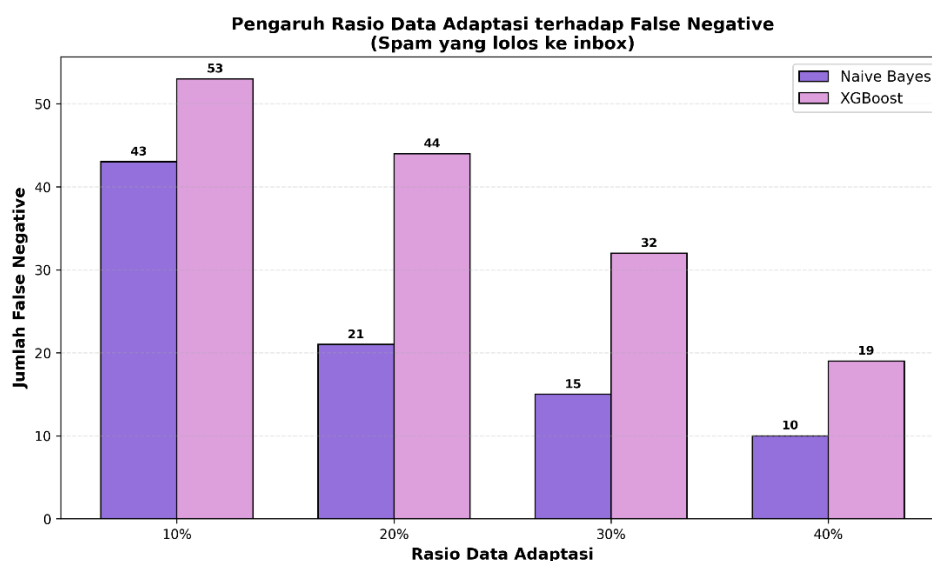


Gambar IV. 11 Eksperimen Rasio *F1-score*

Sumber : (Hasil Penelitian, 2026)

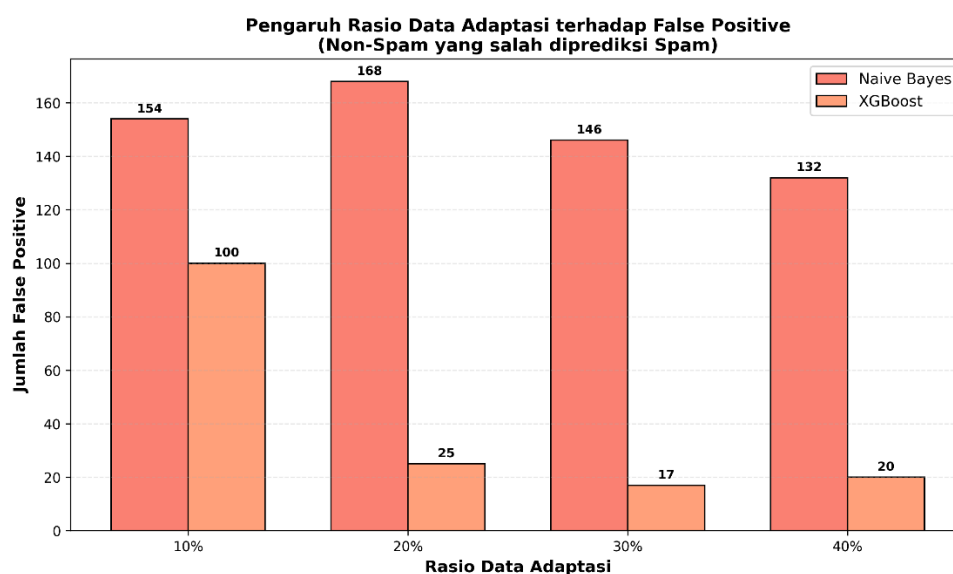
Dari sisi kesalahan klasifikasi, rasio 30% juga menunjukkan keseimbangan yang baik. Pada *XGBoost*, jumlah *false positive* menurun dari 100 pada

rasio 10% menjadi 17 pada rasio 30%. Nilai ini lebih baik dibandingkan rasio 40% yang menghasilkan 20 *false positive*. Sementara itu, jumlah *false negative* *XGBoost* pada rasio 30% adalah 32, lebih rendah dibandingkan rasio 10% dan 20%, meskipun masih lebih tinggi dibandingkan rasio 40%. Dengan demikian, rasio 30% memberikan kombinasi yang seimbang antara akurasi, *F1-score*, jumlah *false positive*, jumlah *false negative*, dan ukuran data uji *final*.



Gambar IV. 12 Eksperimen Rasio *FalsePositive*

Sumber : (Hasil Penelitian, 2026)



Gambar IV. 13 Eksperimen Rasio *FalseNegative*

Sumber : (Hasil Penelitian, 2026)

Berdasarkan hasil tersebut, proporsi 30% dipilih sebagai rasio adaptasi utama dalam penelitian ini. Meskipun rasio 40% menghasilkan akurasi *XGBoost* sedikit lebih tinggi, selisih peningkatannya hanya 0,50 poin persentase, sedangkan jumlah data uji final berkurang dari 700 menjadi 600 *email*. Oleh karena itu, rasio 30% dinilai lebih tepat karena tetap memberikan performa *XGBoost* yang tinggi sebesar 93,00%, sekaligus mempertahankan 700 *email* sebagai data uji *final* yang lebih luas untuk mengevaluasi kemampuan generalisasi model.

4.2.5. Pembahasan Dampak *Domain Adaptation* terhadap *Concept Drift*

Melalui kompilasi uji coba eksperimental, parameter evaluasi, dan grafik hasil yang telah sajikan, serangkaian analisis kausalitas yang komprehensif dapat dipetakan. Secara keseluruhan, integrasi hasil penelitian ini berfokus pada kelima implikasi strategis berikut:

1. Konfirmasi Terjadinya *Domain Mismatch*: Penurunan akurasi hingga tersisa 48,20% pada skenario Metode 1 menjadi indikasi kuat atas eksistensi *Concept Drift*. Model yang hanya dilatih pada data historis karakteristik kosakata masa lampau akan langsung mengalami penurunan kemampuan generalisasi saat dieksekusi melawan distribusi kosa kata dinamis milik ancaman modern.
2. Keberhasilan Restorasi Melalui *Domain Adaptation*: Strategi penyesuaian domain latih dengan menyuntikkan sampel dari wilayah target, serta mendorongnya dengan bobot sampel (*Instance Weighting*) 8,0x, menghasilkan peningkatan performa yang stabil. Model dengan sukses lebih mampu menyesuaikan diri pada data masa lalu dan kembali belajar mendeteksi fitur dari pergeseran yang nyata.
3. Keunggulan *XGBoost*: *XGBoost* secara konsisten mengungguli *Complement Naive Bayes* dengan capaian akurasi final 93,00%. Hal ini terjadi karena arsitektur berbasis *Gradient Boosting trees* milik *XGBoost* mampu menangkap interaksi fitur

hybrid non-linear yang kompleks antara vektor TF-IDF dan fitur heuristik kustom, sebuah kemampuan kalkulasi yang tidak dimiliki oleh pendekatan probabilistik milik *Complement Naive Bayes*.

4. Analisis Rasio *False Positive*: Meskipun algoritma *Complement Naive Bayes* dapat memperbaiki kemampuannya dari 51,50% menjadi 77,00% sebagai *baseline* riset, tingginya angka *False Positive* (146 *email* sah salah diklasifikasikan sebagai *spam*) membuatnya tidak aman untuk diimplementasikan secara praktis. Keunggulan *XGBoost* terletak pada kemampuannya menjaga keseimbangan metrik (*F1-score* 93,00%) dengan hanya memproduksi 17 *False Positive*, menjamin fungsionalitas penyaringan yang aman bagi kotak masuk personal.
5. Kesimpulan Terhadap Rumusan Masalah: Pada akhirnya, rekayasa eksperimen yang telah dilakukan secara solid menjawab seluruh elemen di dalam rumusan masalah. Kinerja klasifikasi lintas-domain tanpa adaptasi dipastikan rendah (kegagalan memproses *Concept Drift*). Metode *Domain Adaptation* dipastikan sukses mengangkat kembali efisiensi tersebut ke dalam garis operasional normal; dan *XGBoost* ditetapkan sebagai arsitektur fundamental terbaik untuk mendukung klasifikasi *spam email* modern. Dengan demikian, Hipotesis Nol (H_0) ditolak dan Hipotesis (H_1) diterima berdasarkan hasil pengujian.

Berdasarkan keseluruhan hasil pengujian, dapat disimpulkan bahwa model yang dilatih hanya menggunakan *Dataset* historis belum mampu melakukan generalisasi dengan baik terhadap *Email* modern. Hal ini terlihat dari rendahnya performa pada Metode 1, baik pada *Complement Naive Bayes* maupun *XGBoost*. Setelah diterapkan *Domain Adaptation* dengan penambahan 30% data primer dan pembobotan instance weight sebesar 8,0, performa kedua algoritma mengalami peningkatan. *Complement Naive Bayes* meningkat menjadi 77,00%, sedangkan

XGBoost mencapai performa terbaik dengan akurasi 93,00%. Dengan demikian, hasil pengujian ini menjawab rumusan masalah bahwa *Domain Adaptation* berperan penting dalam mengurangi dampak domain mismatch dan *Concept Drift* pada klasifikasi *spam Email* modern, dengan *XGBoost* sebagai model yang memberikan performa paling baik pada skenario pengujian ini.

4.3. Implementasi Model dalam Bentuk Demo Aplikasi *Web* Lokal

Sebagai bentuk implementasi tambahan dari hasil penelitian yang telah dilakukan, model klasifikasi *spam email* yang dikembangkan pada penelitian ini diintegrasikan ke dalam sebuah aplikasi *web* lokal berbasis *Flask*. Implementasi ini bertujuan untuk mendemonstrasikan kemampuan model dalam melakukan klasifikasi *spam email* secara langsung melalui antarmuka yang lebih interaktif dan mudah digunakan.

Model yang diimplementasikan pada aplikasi merupakan model *Complement Naive Bayes* dan *Extreme Gradient Boosting (XGBoost)* yang telah melalui proses pelatihan menggunakan *Dataset* historis serta pendekatan *Domain Adaptation* pada data *primer*. Aplikasi dijalankan pada lingkungan lokal (*localhost*) dan berfungsi sebagai media simulasi untuk menguji kemampuan model dalam melakukan prediksi terhadap *email* baru yang dimasukkan oleh pengguna.

Implementasi aplikasi *web* ini menunjukkan bahwa model hasil penelitian dapat diterapkan ke dalam sistem sederhana yang mampu melakukan klasifikasi *spam email* secara langsung. Namun demikian, aplikasi *web* ini tidak menjadi objek utama penelitian dan tidak digunakan sebagai dasar pengambilan kesimpulan penelitian.

4.3.1. Tujuan Pengembangan Demo Aplikasi

Pengembangan demo aplikasi *web* lokal pada penelitian ini bertujuan untuk memberikan gambaran implementasi praktis dari model klasifikasi *spam email* yang telah dibangun. Aplikasi ini digunakan sebagai sarana demonstrasi untuk menunjukkan bahwa model hasil pelatihan dapat digunakan secara langsung dalam melakukan prediksi terhadap *email* baru yang dimasukkan oleh pengguna.

Demo aplikasi ini digunakan sebagai media simulasi untuk menguji model pada data *email* yang belum pernah digunakan sebelumnya. Dengan demikian, pengguna dapat memperoleh gambaran mengenai cara kerja model klasifikasi yang telah dikembangkan tanpa harus menjalankan proses pelatihan dan pengujian secara langsung melalui kode program.

Pengembangan demo aplikasi ini tidak menjadi fokus utama penelitian dan tidak digunakan sebagai dasar evaluasi performa model. Seluruh analisis dan kesimpulan penelitian tetap mengacu pada hasil pengujian yang telah dilakukan pada skenario eksperimen yang dijelaskan pada subbab sebelumnya.

4.3.2. Lingkungan Implementasi Demo Aplikasi

Demo aplikasi pada penelitian ini dikembangkan menggunakan bahasa pemrograman Python dengan memanfaatkan *framework Flask* sebagai penghubung antara antarmuka pengguna dan model klasifikasi yang telah dilatih sebelumnya. Aplikasi dijalankan pada lingkungan lokal (*localhost*) sehingga seluruh proses prediksi dapat dilakukan secara mandiri tanpa memerlukan koneksi ke layanan server eksternal.

Aplikasi dikembangkan menggunakan HTML, CSS, dan JavaScript untuk menampilkan hasil prediksi secara interaktif. Selain itu, aplikasi juga memanfaatkan berbagai pustaka Python untuk mendukung proses *Preprocessing*, ekstraksi fitur TF-IDF, pengelolaan data, serta visualisasi hasil evaluasi model.

Dengan lingkungan implementasi tersebut, aplikasi mampu menjalankan proses klasifikasi *spam email* secara lokal dan menampilkan hasil prediksi dalam bentuk yang lebih mudah dipahami oleh pengguna.

4.3.3. Fitur dan Fungsionalitas Demo Aplikasi

Demo aplikasi yang dikembangkan pada penelitian ini menyediakan beberapa fitur untuk mempermudah proses klasifikasi *spam email* dan visualisasi hasil prediksi model. Fitur-fitur tersebut dirancang untuk mendemonstrasikan kemampuan model *Complement Naive Bayes* dan *Extreme Gradient Boosting* (*XGBoost*) yang telah dilatih menggunakan metode tanpa adaptasi maupun pendekatan *Domain Adaptation*.

Salah satu fitur utama yang tersedia adalah klasifikasi *email* berbasis teks, di mana pengguna dapat memasukkan isi *email* secara langsung melalui antarmuka aplikasi. Sistem kemudian melakukan proses *Preprocessing*, ekstraksi fitur, dan klasifikasi untuk menghasilkan prediksi *spam* atau *non-spam*.

Selain prediksi berbasis teks, aplikasi juga menyediakan fitur evaluasi menggunakan berkas CSV yang memungkinkan pengguna melakukan pengujian terhadap banyak data *email* secara sekaligus. Melalui fitur ini, sistem dapat menampilkan hasil evaluasi model dalam bentuk metrik akurasi, presisi, *recall*, dan *F1-score*.

Aplikasi juga menampilkan informasi tambahan berupa probabilitas prediksi, tingkat kepercayaan (*confidence score*), nilai *threshold* yang digunakan, serta hasil keputusan akhir model. Informasi tersebut bertujuan untuk memberikan gambaran yang lebih jelas mengenai proses pengambilan keputusan yang dilakukan oleh model klasifikasi.

Untuk mendukung analisis hasil, aplikasi menyediakan visualisasi *Confusion matrix*, performa masing-masing algoritma, serta perbandingan hasil antara metode tanpa adaptasi dan metode dengan *Domain Adaptation*. Dengan adanya fitur tersebut, pengguna dapat memahami pengaruh penerapan *Domain Adaptation* terhadap peningkatan performa model secara lebih mudah.

Secara keseluruhan, fitur-fitur yang tersedia pada demo aplikasi berfungsi sebagai sarana demonstrasi implementasi model hasil penelitian dan bukan sebagai bagian utama dari proses evaluasi yang digunakan dalam penarikan kesimpulan penelitian.

4.3.4. Alur Kerja Demo Aplikasi

Alur kerja demo aplikasi dimulai ketika pengguna memasukkan data *email* melalui antarmuka yang tersedia, baik dalam bentuk teks tunggal maupun berkas CSV. Data yang diterima kemudian diproses menggunakan tahapan *Preprocessing* yang sama dengan proses pelatihan model pada penelitian ini.

Setelah proses *Preprocessing* selesai dilakukan, sistem melakukan ekstraksi fitur menggunakan metode TF-IDF yang dikombinasikan dengan fitur struktural dan fitur kata kunci *spam*. Hasil ekstraksi fitur tersebut kemudian digunakan sebagai masukan bagi model *Complement Naive Bayes* dan *Extreme Gradient Boosting* (*XGBoost*) yang telah dilatih sebelumnya.

Pada tahap klasifikasi, masing-masing model menghasilkan prediksi berupa kategori *spam* atau *non-spam* beserta nilai probabilitas prediksi. Sistem selanjutnya menerapkan nilai *threshold* yang telah ditentukan untuk menghasilkan keputusan akhir klasifikasi. Hasil prediksi kemudian ditampilkan kepada pengguna melalui antarmuka aplikasi.

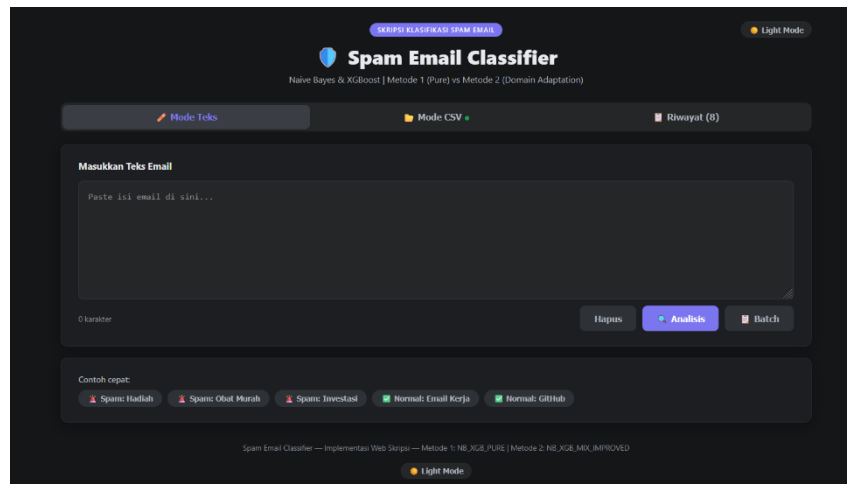
Untuk pengujian menggunakan berkas CSV, sistem akan melakukan proses klasifikasi terhadap seluruh data yang diunggah dan menampilkan hasil evaluasi dalam bentuk metrik performa, *Confusion matrix*, serta perbandingan hasil antar metode yang digunakan. Dengan demikian, pengguna dapat memperoleh gambaran mengenai performa model secara lebih mudah melalui visualisasi yang disediakan aplikasi.

4.3.5. Tampilan Demo Aplikasi

Demo aplikasi *web* yang dikembangkan pada penelitian ini terdiri atas beberapa halaman dan fitur yang digunakan untuk mendemonstrasikan kemampuan model klasifikasi *spam email* hasil penelitian. Seluruh fitur diimplementasikan menggunakan antarmuka berbasis *web* sehingga pengguna dapat melakukan pengujian model secara lebih mudah tanpa harus menjalankan program melalui lingkungan pengembangan.

1. Halaman Utama Aplikasi

Halaman utama merupakan halaman awal yang ditampilkan ketika aplikasi dijalankan. Pada halaman ini tersedia tiga menu utama, yaitu Mode Teks, Mode CSV, dan Riwayat. Selain itu, pengguna juga dapat mengaktifkan tampilan *Dark Mode* untuk meningkatkan kenyamanan penggunaan aplikasi.



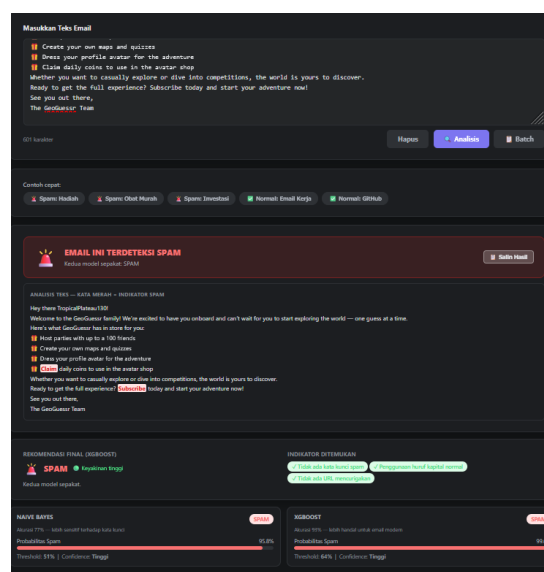
Gambar IV. 14 Halaman Utama Demo Aplikasi

Sumber : (Hasil Penelitian, 2026)

Pada halaman utama, pengguna dapat memilih metode pengujian yang diinginkan sesuai kebutuhan. Struktur antarmuka dirancang sederhana agar proses demonstrasi model dapat dilakukan secara mudah dan intuitif.

2. Tampilan Prediksi *Email* Berbasis Teks

Mode Teks digunakan untuk melakukan klasifikasi terhadap satu *email* secara langsung. Pengguna cukup memasukkan isi *email* ke dalam kolom yang tersedia kemudian menekan tombol Analisis untuk memperoleh hasil prediksi.

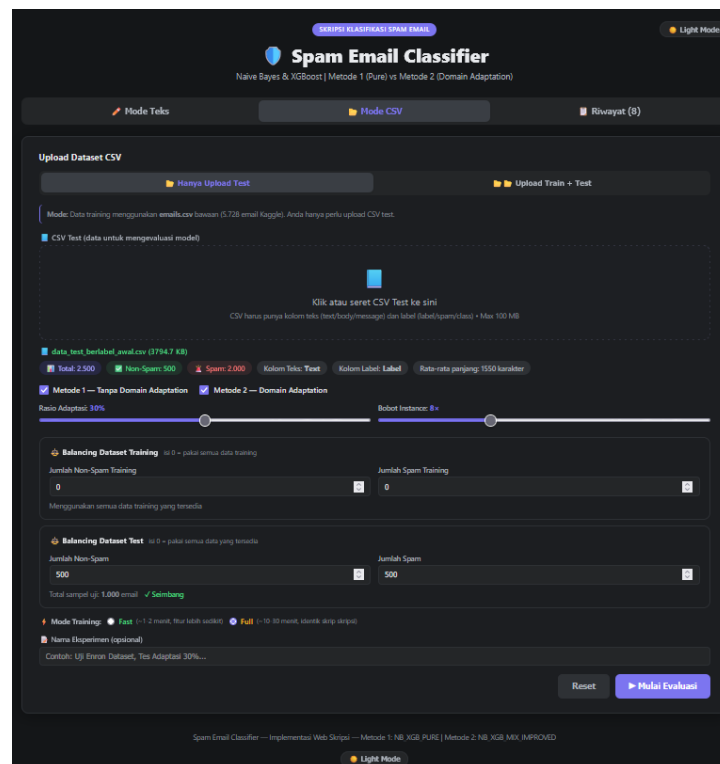
Gambar IV. 15 Tampilan Prediksi *Email* Berbasis Teks

Sumber : (Hasil Penelitian, 2026)

Setelah proses analisis selesai dilakukan, sistem akan menampilkan hasil klasifikasi *spam* atau *non-spam* beserta nilai probabilitas dari model *Complement Naive Bayes* dan *XGBoost*. Sistem juga menampilkan indikator kata kunci yang terdeteksi serta rekomendasi akhir berdasarkan hasil prediksi model yang digunakan.

3. Tampilan Pengujian *Dataset* CSV

Mode CSV digunakan untuk melakukan pengujian model menggunakan *Dataset* dalam format CSV. Pada fitur ini pengguna dapat mengunggah data uji, mengatur rasio adaptasi, menentukan nilai *instance weighting*, serta memilih metode evaluasi yang akan digunakan.



Gambar IV. 16 Tampilan Pengujian *Dataset* CSV

Sumber : (Hasil Penelitian, 2026)

Fitur ini memungkinkan pengguna melakukan simulasi pengujian yang serupa dengan skenario penelitian. Selain itu, sistem juga menyediakan pengaturan jumlah data *spam* dan *non-spam* yang digunakan selama proses evaluasi.

4. Tampilan Proses Pelatihan dan Evaluasi

Setelah *Dataset* berhasil diunggah, aplikasi akan menjalankan proses pelatihan dan evaluasi model sesuai parameter yang dipilih pengguna. Selama proses berlangsung, sistem menampilkan informasi status pelatihan dan progres eksekusi secara *real-time*.

The screenshot displays the application's interface for dataset upload and training. At the top, there are tabs for 'Mode Teks' and 'Mode CSV', and a 'Riwayat' (History) section. The main area is titled 'Upload Dataset CSV' and includes a 'Hanya Upload Test' button. Below this, there is a section for 'CSV Test (data untuk mengevaluasi model)' with a 'Klik atau seret CSV Test ke sini' instruction. A file named 'data_test_berlabel_awal.csv' (3794.7 KB) is shown with details: Total: 2,500, Non-Spam: 500, Spam: 2,000, Kolom Teks: Text, Kolom Label: Label, and Rata-rata panjang: 1550 karakter. The interface also shows 'Metode 1 - Tanpa Domain Adaptation' and 'Metode 2 - Domain Adaptation' with a 'Ratio Adaptasi: 30%' slider. Below this, there are sections for 'Balancing Dataset Training' and 'Balancing Dataset Test' with input fields for 'Jumlah Non-Spam Training' and 'Jumlah Spam Training'. The 'Balancing Dataset Test' section shows 'Jumlah Non-Spam: 500' and 'Jumlah Spam: 500' with a 'Total sampel uji: 1,000 email' and a 'Seimbang' status. At the bottom, there is a 'Mode Training' section with 'Fast' and 'Full' options, and a 'Nama Eksperimen (optional)' field. A 'Reset' button and a 'Mulai Evaluasi' button are also present. A progress bar for 'Memproses...' is shown at the bottom, and a terminal window displays the execution of training and evaluation commands.

Gambar IV. 17 Tampilan Proses Pelatihan dan Evaluasi

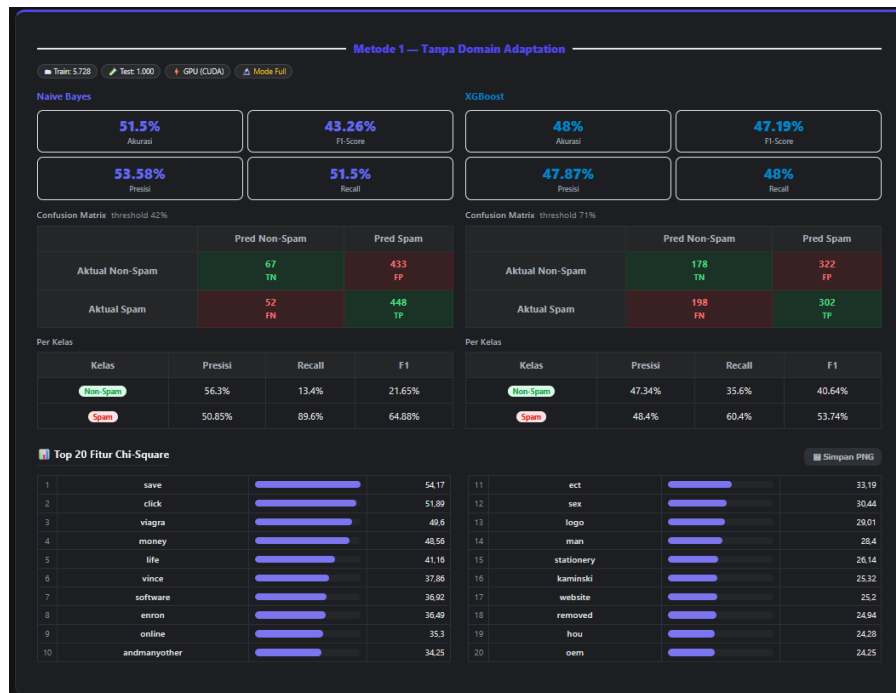
Sumber : (Hasil Penelitian, 2026)

Informasi yang ditampilkan meliputi jumlah data pelatihan, jumlah data pengujian, proses ekstraksi fitur, proses pelatihan model, serta tahapan evaluasi yang sedang dijalankan.

5. Tampilan Hasil Evaluasi Model

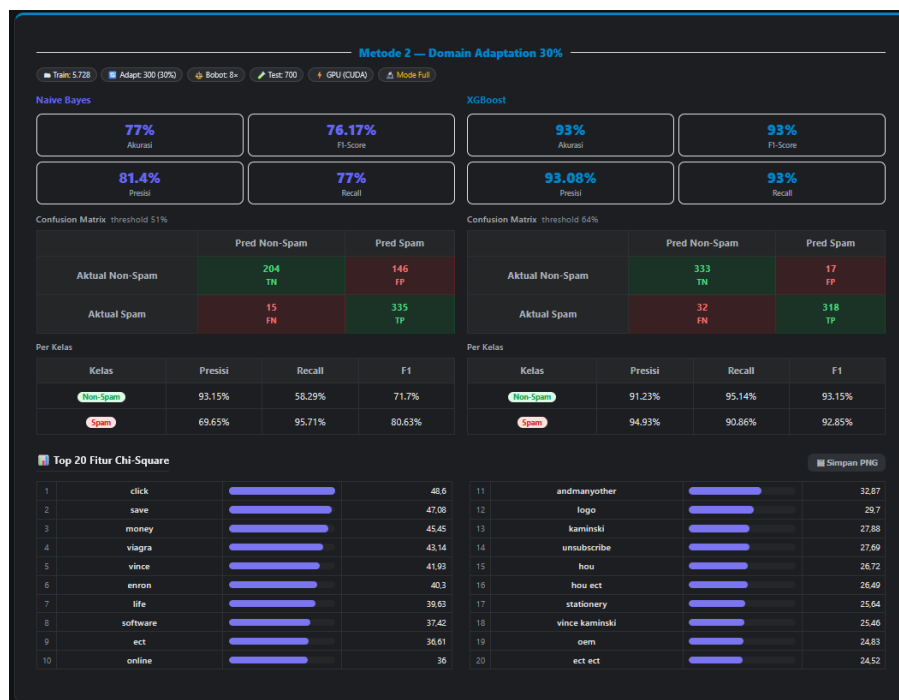
Setelah proses evaluasi selesai dilakukan, sistem akan menampilkan hasil performa masing-masing model pada Metode 1 dan Metode 2. Informasi yang

ditampilkan meliputi nilai akurasi, presisi, *recall*, *F1-score*, *Confusion matrix*, serta daftar fitur dengan nilai *Chi-Square* tertinggi.



Gambar IV. 18 Tampilan Hasil Evaluasi Metode 1

Sumber : (Hasil Penelitian, 2026)



Gambar IV. 19 Tampilan Hasil Evaluasi Metode 2

Sumber : (Hasil Penelitian, 2026)

Hasil evaluasi disajikan dalam bentuk visual sehingga memudahkan pengguna untuk membandingkan performa model *Complement Naive Bayes* dan *XGBoost* pada kedua metode pengujian yang digunakan dalam penelitian.

6. Tampilan Perbandingan Metode

Aplikasi juga menyediakan fitur perbandingan hasil evaluasi antara Metode 1 dan Metode 2. Perbandingan dilakukan berdasarkan metrik akurasi, presisi, *recall*, dan *F1-score*.



Model	Akurasi			Presisi			Recall			F1-Score		
	M1	M2	Δ	M1	M2	Δ	M1	M2	Δ	M1	M2	Δ
Naive Bayes	51.5%	77%	▲ +25.50%	53.58%	81.4%	▲ +27.82%	51.5%	77%	▲ +25.50%	43.26%	76.17%	▲ +32.91%
XGBoost	48%	93%	▲ +45.00%	47.87%	93.08%	▲ +45.21%	48%	93%	▲ +45.00%	47.19%	93%	▲ +45.81%

M1 = Tanpa Domain Adaptation • M2 = Domain Adaptation 30% • Δ = selisih M2 terhadap M1

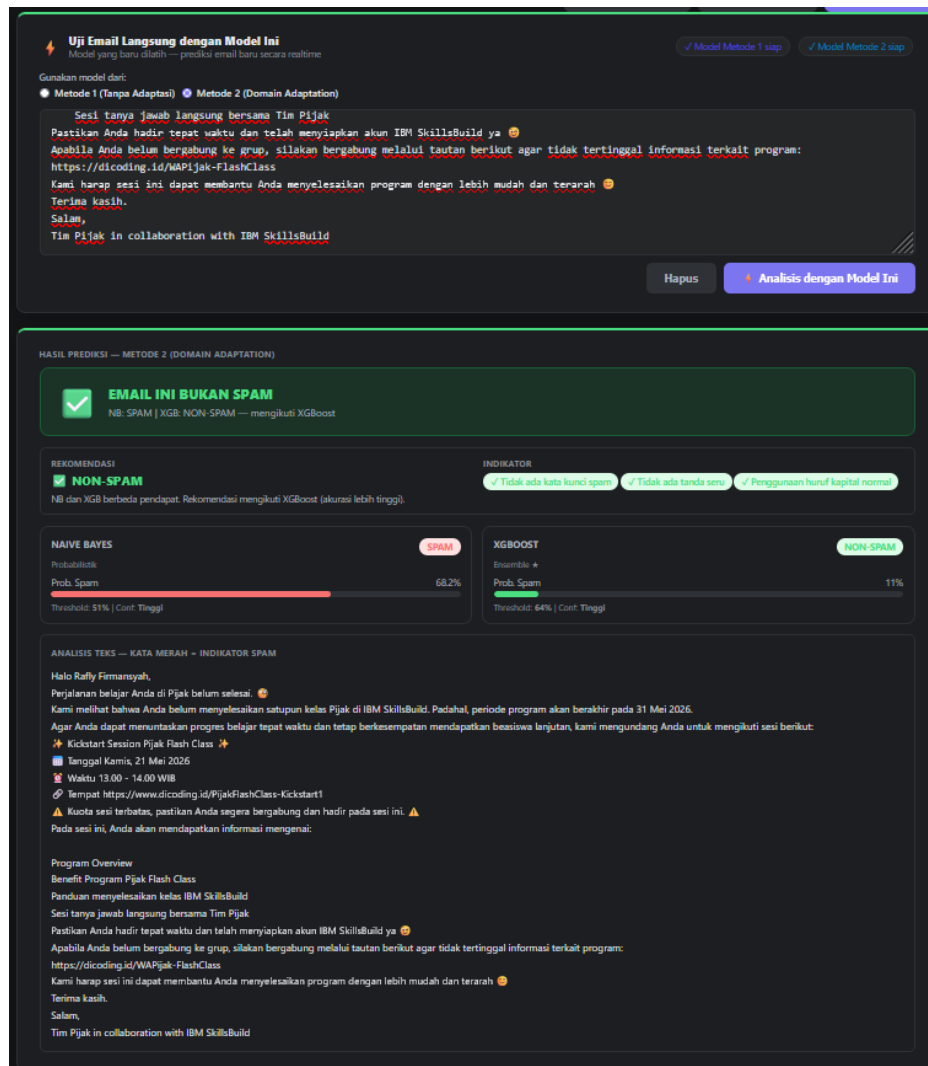
Simpan JSON Simpan CSV Print / PDF

Gambar IV. 20 Tampilan Perbandingan Metode 1 dan Metode 2
Sumber : (Hasil Penelitian, 2026)

Melalui fitur ini, pengguna dapat melihat secara langsung peningkatan performa yang diperoleh setelah penerapan *Domain Adaptation* dan teknik *Instance Weighting* pada model klasifikasi.

7. Tampilan Pengujian Model Hasil Pelatihan

Setelah proses pelatihan selesai dilakukan, model yang telah terbentuk dapat langsung digunakan untuk mengklasifikasikan *email* baru melalui fitur pengujian model. Pengguna dapat memasukkan isi *email* dan memilih model yang akan digunakan untuk melakukan prediksi.



Gambar IV. 21 Tampilan Pengujian Model Hasil Pelatihan

Sumber : (Hasil Penelitian, 2026)

Fitur ini digunakan untuk menunjukkan bahwa model hasil penelitian dapat dimanfaatkan secara langsung untuk melakukan klasifikasi *email* baru tanpa perlu melakukan proses pelatihan ulang.

8. Tampilan Riwayat Eksperimen

Aplikasi menyediakan fitur riwayat eksperimen yang berfungsi untuk menyimpan hasil pengujian yang telah dilakukan. Informasi yang tersimpan meliputi waktu pengujian, parameter yang digunakan, serta hasil performa masing-masing model.



Gambar IV. 22 Tampilan Riwayat Eksperimen

Sumber : (Hasil Penelitian, 2026)

Selain menyimpan riwayat pengujian, aplikasi juga menyediakan visualisasi grafik performa dan fitur ekspor hasil ke dalam format CSV sehingga memudahkan pengguna dalam melakukan dokumentasi hasil eksperimen.

4.3.6. Pembahasan Implementasi Demo Aplikasi

Implementasi model klasifikasi *spam email* ke dalam bentuk demo aplikasi berbasis *web* lokal menunjukkan bahwa model yang dihasilkan dari penelitian ini dapat diterapkan pada lingkungan yang lebih interaktif dan mudah digunakan oleh pengguna. Melalui aplikasi yang dikembangkan menggunakan *framework Flask*,

seluruh proses klasifikasi mulai dari masukan data, proses *Preprocessing*, ekstraksi fitur, prediksi model, hingga penyajian hasil dapat dilakukan secara otomatis melalui antarmuka *web*.

Berdasarkan implementasi yang telah dilakukan, aplikasi mampu menjalankan dua skenario pengujian yang sesuai dengan penelitian, yaitu Metode 1 (Tanpa *Domain Adaptation*) dan Metode 2 (*Domain Adaptation*). Hasil yang ditampilkan pada aplikasi juga konsisten dengan hasil eksperimen yang telah diperoleh pada pengujian utama penelitian. Pada skenario Metode 2, model *XGBoost* tetap menunjukkan performa terbaik dibandingkan *Complement Naive Bayes* dengan tingkat akurasi yang lebih tinggi pada data *email* modern.

Selain menampilkan hasil klasifikasi *email* tunggal melalui Mode Teks, aplikasi juga menyediakan fasilitas evaluasi *Dataset* dalam format CSV sehingga pengguna dapat melakukan simulasi pengujian menggunakan data dalam jumlah besar. Fitur tambahan seperti pengaturan rasio adaptasi, instance weighting, visualisasi *Confusion matrix*, grafik perbandingan performa, serta penyimpanan riwayat eksperimen membantu pengguna memahami pengaruh penerapan *Domain Adaptation* terhadap performa model secara lebih jelas.

Implementasi ini juga membuktikan bahwa model hasil penelitian dapat digunakan untuk melakukan klasifikasi *email* baru secara langsung tanpa perlu melakukan proses pelatihan ulang. Dengan demikian, model yang dihasilkan tidak hanya memiliki performa yang baik pada tahap eksperimen, tetapi juga dapat diterapkan pada aplikasi sederhana sebagai media demonstrasi maupun simulasi penggunaan.

Meskipun demikian, demo aplikasi yang dikembangkan masih dijalankan pada lingkungan lokal (*localhost*) dan belum dirancang untuk penggunaan pada

lingkungan produksi. Aspek seperti keamanan sistem, skalabilitas, autentikasi pengguna, integrasi dengan layanan *email*, serta optimasi performa server belum menjadi fokus pada penelitian ini. Oleh karena itu, implementasi aplikasi *web* hanya digunakan sebagai sarana demonstrasi untuk menunjukkan penerapan model hasil penelitian dan tidak digunakan sebagai dasar utama dalam pengambilan kesimpulan penelitian.

BAB V

PENUTUP

5.1. Kesimpulan

Berdasarkan seluruh tahapan penelitian yang meliputi pengumpulan data, rekayasa fitur, implementasi *Domain Adaptation*, pelatihan model, serta evaluasi performa menggunakan metrik akurasi, presisi, *recall*, dan *F1-score*, maka dapat diperoleh beberapa kesimpulan sebagai berikut:

1. Fenomena *Concept Drift* terbukti memberikan dampak yang signifikan terhadap penurunan performa model klasifikasi *spam email* yang dilatih menggunakan data historis. Hasil pengujian pada Metode 1 (Tanpa Adaptasi) menunjukkan bahwa algoritma *Complement Naive Bayes* dan *Extreme Gradient Boosting* (*XGBoost*) yang dilatih menggunakan *Dataset* sekunder historis tidak mampu melakukan generalisasi secara optimal terhadap data *email* personal modern. Hal ini ditunjukkan oleh nilai akurasi sebesar 51,50% pada *Complement Naive Bayes* dan 48,20% pada *XGBoost*, yang mengindikasikan rendahnya kemampuan model dalam menghadapi perbedaan distribusi data antara domain pelatihan dan domain pengujian.
2. Penerapan metode *Supervised Domain Adaptation* melalui skema injeksi 30% data primer ke dalam data pelatihan, yang diperkuat dengan teknik *Instance Weighting* sebesar 8 kali lipat pada data adaptasi, terbukti efektif dalam meningkatkan performa model klasifikasi. Pendekatan tersebut mampu membantu model mengenali karakteristik kosakata dan pola *email* modern secara lebih baik tanpa menghilangkan pengetahuan yang telah diperoleh dari data historis.
3. Algoritma *Extreme Gradient Boosting* (*XGBoost*) menunjukkan performa terbaik pada skenario *Domain Adaptation* (Metode 2). Model ini berhasil mencapai akurasi

sebesar 93,00%, presisi sebesar 93,08%, *recall* sebesar 93,00%, dan *F1-score* sebesar 93,00%. Selain itu, jumlah *False Positive* yang dihasilkan relatif rendah, yaitu 17 *email* dari total 350 *email non-spam*. Sementara itu, *Complement Naive Bayes* juga mengalami peningkatan performa hingga mencapai akurasi sebesar 77,00%, namun masih menghasilkan jumlah *False Positive* yang lebih tinggi dibandingkan *XGBoost*. Berdasarkan hasil tersebut, *XGBoost* dapat dinyatakan sebagai algoritma yang paling optimal untuk klasifikasi *spam email* lintas domain pada penelitian ini.

4. Hasil penelitian menunjukkan bahwa penggunaan *Dataset* historis saja tidak cukup untuk membangun model klasifikasi *spam email* yang mampu beradaptasi terhadap karakteristik *email* modern. Oleh karena itu, proses pembaruan data pelatihan atau penerapan *Domain Adaptation* menjadi faktor penting untuk mempertahankan performa model ketika digunakan pada lingkungan yang memiliki distribusi data berbeda dari data pelatihan awal.

5.2. Saran

Berdasarkan batasan masalah dan hasil penelitian yang telah diperoleh, terdapat beberapa saran yang dapat dijadikan acuan untuk penelitian selanjutnya sebagai berikut:

1. Implementasi Sistem Secara *Real-time*

Penelitian ini berfokus pada evaluasi performa model klasifikasi *spam email* menggunakan data statis dalam lingkungan pengujian *offline*. Oleh karena itu, penelitian selanjutnya dapat mengimplementasikan model dengan performa terbaik, yaitu *XGBoost*, ke dalam sistem yang berjalan secara *real-time*, seperti *Application Programming Interface (API)*, layanan berbasis *web*, maupun ekstensi

klien *email*. Pengujian tersebut dapat digunakan untuk mengukur performa model dari sisi kecepatan inferensi, penggunaan sumber daya komputasi, serta stabilitas sistem pada lingkungan operasional yang sebenarnya.

2. Penerapan Pendekatan *Online Learning*

Fenomena *Concept Drift* yang menjadi fokus penelitian ini menunjukkan bahwa karakteristik *spam email* dapat berubah seiring waktu. Oleh karena itu, penelitian selanjutnya dapat menerapkan pendekatan *Online Learning* atau *Incremental Learning* agar model mampu melakukan pembaruan pengetahuan secara bertahap tanpa perlu dilatih ulang dari awal. Pendekatan ini diharapkan dapat meningkatkan kemampuan adaptasi model terhadap pola *spam* baru yang terus berkembang.

3. Pengujian pada *Dataset* Multibahasa

Dataset yang digunakan dalam penelitian ini masih didominasi oleh *email* berbahasa Inggris. Penelitian selanjutnya dapat memperluas cakupan pengujian dengan menggunakan *Dataset* multibahasa, seperti bahasa Indonesia maupun kombinasi beberapa bahasa lainnya. Pengujian tersebut dapat digunakan untuk mengevaluasi kemampuan *Domain Adaptation* dan teknik ekstraksi fitur dalam menangani perbedaan karakteristik bahasa yang lebih kompleks.

4. Pengembangan dan Evaluasi Metode *Domain Adaptation* Lainnya

Penelitian ini menggunakan pendekatan *Supervised Domain Adaptation* yang dikombinasikan dengan teknik *Instance Weighting*. Pada penelitian selanjutnya, dapat dilakukan *perbandingan* dengan metode *Domain Adaptation* lainnya, seperti *Transfer Learning*, *Adversarial Domain Adaptation*, maupun *Semi-Supervised Domain Adaptation*. Tujuannya adalah untuk mengetahui pendekatan

yang paling efektif dalam mengatasi pergeseran distribusi data pada kasus klasifikasi *spam email*.

5. Pengembangan Implementasi Sistem

Meskipun penelitian ini berfokus pada evaluasi performa model klasifikasi spam email, hasil *penelitian* telah berhasil diimplementasikan dalam bentuk demo aplikasi *web* lokal berbasis *Flask*. Oleh karena itu, penelitian selanjutnya dapat mengembangkan aplikasi tersebut menjadi sistem yang lebih komprehensif dengan integrasi ke layanan email nyata, penyimpanan basis data, mekanisme autentikasi pengguna, serta kemampuan klasifikasi secara *real-time*. Pengembangan tersebut dapat digunakan untuk menguji kesiapan model pada lingkungan operasional yang sesungguhnya.